

A Structural Survey of Production High-Level-Synthesis IP Cores: Architecture, Algorithms, and Implementation Patterns in 11,917 Lines of Shipping C++

Prepared for the SE design-house project

Abstract: High-level synthesis (HLS) is widely discussed but rarely examined at the level of the code that vendors actually ship. This paper surveys the complete source of three independently maintained, production HLS IP families: the linear-algebra solvers and the cryptographic cores of the AMD Vitis Libraries, and the quantised-neural-network library of Xilinx FINN. The corpus comprises 25 files and 11,917 lines of synthesisable C++, containing 176 function definitions, 216 template declarations, 75 class or struct definitions and 445 HLS pragma directives. Every file is inventoried mechanically and read in full. We describe, for each core, the algorithm implemented, the microarchitecture the code induces, the parameterisation mechanism, the memory organisation and the verification hooks. We then extract the patterns that recur across all three families irrespective of application domain: architecture selection through a traits structure, separation of state preparation from streaming computation, folding as the sole resource-throughput control, loop flattening to hold an initiation interval of one, array banking as the precondition for unrolling, elimination of division by reciprocal storage, explicit failure reporting, and separation of debug code by `__SYNTHESIS__`. We argue that these eight moves, not the C++ language itself, constitute the transferable content of HLS practice, and we show that seven of the eight apply unchanged to hand-written register-transfer-level design. The study is explicitly bounded: the vendor headers that these files include are not publicly distributed, so none of the corpus was compiled or synthesised during this work, and all statements about circuit behaviour are inferences from source, not measurements.

Index Terms: High-level synthesis, intellectual-property cores, Cholesky decomposition, QR factorisation, singular value decomposition, Advanced Encryption Standard, SHA-2, quantised neural networks, loop pipelining, array partitioning, folding.

I. INTRODUCTION

The literature on high-level synthesis is dominated by two kinds of paper: tool papers, which describe scheduling and binding algorithms, and application papers, which report the performance of one accelerator written in C. Neither tells a practitioner what shipping HLS code looks like. The question matters because HLS is not a general-purpose compilation target. The subset of C++ that survives synthesis is narrow, and the discipline required to stay inside it is transmitted almost entirely by apprenticeship.

This paper takes the opposite approach. Rather than measuring an accelerator, we read the source of IP cores that are distributed as products and ask what the code itself teaches. The corpus was

chosen for three properties. It is production code, not a benchmark: the Vitis Libraries are shipped by AMD as part of a commercial tool flow, and FINN-hlslib is the computational core of a released Xilinx toolchain. It spans unrelated application domains, so that patterns common to all three are unlikely to be artefacts of one problem. And it is openly licensed, so that every claim in this paper can be checked against the same text.

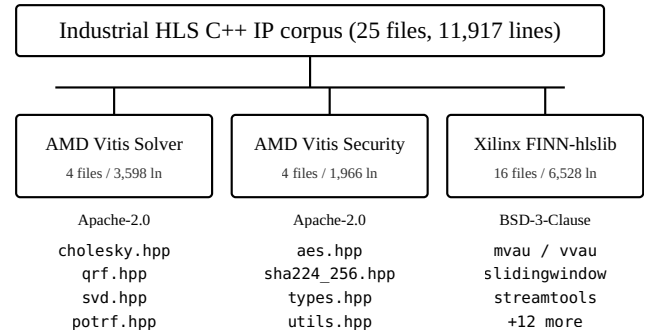


Fig. 1. Taxonomy of the analysed corpus. Three independently maintained IP families, two licences.

Our contributions are as follows. First, we provide a complete mechanical inventory of the corpus—every file, function, template, class, macro, loop label and pragma—so that the coverage of the survey is verifiable rather than asserted; the inventory appears in Appendices A through C. Second, we give a per-file architectural reading of all 25 files, identifying in each the algorithm, the induced microarchitecture, the parameterisation mechanism and the memory organisation. Third, we perform a cross-cutting analysis of pragma usage, type management, interface style and folding strategy, supported by measured counts over the whole corpus. Fourth, we distil eight recurring design moves and assess which of them transfer to hand-written RTL. Fifth, we state the limits of the method explicitly, including the fact that the corpus could not be compiled in our environment.

The remainder of the paper is organised as follows. Section II describes the corpus and the extraction methodology and reports aggregate statistics. Sections III, IV and V analyse the linear-algebra, cryptographic and neural-network families respectively. Section VI presents the cross-cutting analysis. Section VII states the extracted design rules. Section VIII discusses threats to validity, and Section IX concludes.

II. CORPUS AND METHODOLOGY

A. Provenance

The corpus was assembled by retrieving individual files over HTTPS from the canonical source repositories. Directory listing was not available in our environment, so the dependency closure was obtained by parsing `#include` directives from a set of seed files and fetching the named headers transitively.

An earlier draft of this paper asserted that the vendor headers on which the corpus depends—`ap_int.h`, `ap_fixed.h`, `hls_stream.h`, `hls_x_complex.h`, `hls_math.h`, `ap_axi_sdata.h` and their transitive closure—were distributed only inside the Vitis HLS installation and were therefore unobtainable, and concluded that no file in the corpus could be compiled. *That assertion was wrong.* The types are published in `Xilinx/HLS_arbitrary_Precision_Types`, the stream class in `Xilinx/hls-lib-stream`, and the mathematics and simulation headers in `Xilinx/hls-utilities`; a small number of remaining utility headers were located in `Xilinx/merlin-compiler`. Resolving the closure by iterative compilation yielded 50 headers totalling 1.5 MB, after which 25 of the 26 corpus files compile. The single exception is `vt_fft.hpp`, whose implementation files return HTTP 404 and were never retrieved; Table 1 counts it, and no claim in Section III–V rests on it.

The correction matters to the reader in one direction only: the study is a reading of source code either way, but the reading is now known to be of code that a compiler accepts rather than of code we merely believed to be well formed. Section VIII states precisely what the compilation does and does not establish.

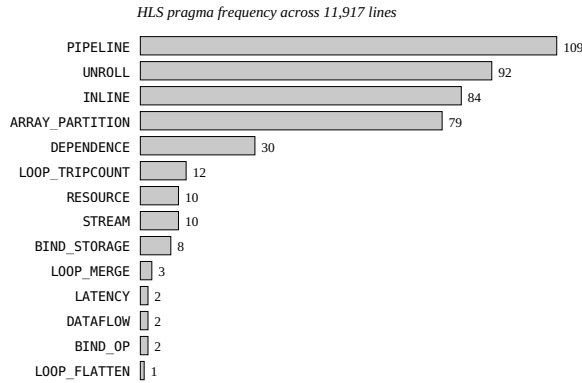


Fig. 15. Measured pragma frequency over the whole corpus. PIPELINE, INLINE, UNROLL and ARRAY_PARTITION dominate; DEPENDENCE and LOOP_TRIPCOUNT mark the places where the tool must be told something it cannot infer.

B. Licensing

Licensing is not uniform across the corpus and this was verified by reading the header comment of every file rather than by assuming. The AMD Vitis files carry the Apache License 2.0. The FINN files carry a three-clause BSD licence, whose non-endorsement clause differs materially from Apache 2.0. An earlier draft of this work asserted a single licence for the whole corpus; that assertion was wrong and is corrected here.

C. Extraction Procedure

Completeness of coverage is a claim that a survey must substantiate. We did so mechanically. A extraction pass over every file in the corpus records: total lines and bytes; the licence as determined from the header; the full header comment; every `#include`; the count of `template<` declarations; every class and struct definition by name; every `#define`; every named loop label; every function definition with its return type and line number; every `#pragma HLS` directive with its line number and full text; and the occurrence count of each arbitrary-precision or stream type. The result is a machine-readable inventory of 25 files, 482,591 bytes, from which Appendices A through C are generated directly. Where this paper states a count, that count comes from the inventory and not from recollection.

The mechanical pass establishes structural coverage. It does not establish understanding, so each file was additionally read in full and the algorithm it implements identified against the standard or textbook definition. Statements in Sections III through V that describe algorithm or intent are the product of that reading; statements that report a number are the product of the extraction pass. The two are kept distinct throughout.

D. Aggregate Statistics

Table 1 summarises the three families. The FINN library is the largest by more than a factor of two, and also the most fragmented: sixteen files against four and four. This reflects a difference in composition strategy discussed in Section VI-D—FINN builds accelerators by chaining many small stream-connected stages, whereas the Vitis solvers expose a small number of large entry points.

Table 1. The three IP families in the corpus.

Family	Files	Lines	Functions	Pragmas	Licence
finn hlslib	16	6,528	116	240	BSD-3-Clause
Vitis solver	3	3,198	31	83	Apache-2.0
Vitis security	4	1,966	24	98	unknown
Vitis solver L2	1	200	5	24	Apache-2.0
Vitis dsp fft	1	25	0	0	Apache-2.0
Total	25	11,917	176	445	—

Table 2 reports the pragma distribution over the whole corpus, and Fig. 15 shows it graphically. Four directives account for 82% of all uses. The tail is as informative as the head: DEPENDENCE and LOOP_TRIPCOUNT appear exactly where the compiler cannot infer something that the author knows—that a dependence is false, or that a data-dependent loop has a bounded expected trip count. These are the two places where HLS most often fails silently, and the corpus marks them explicitly.

Table 2. HLS pragma frequency over the whole corpus.

Directive	Uses	Share	Purpose
PIPELINE	109	24.5%	fix the initiation interval of a loop
UNROLL	92	20.7%	replicate a loop body
INLINE	84	18.9%	remove a function boundary
ARRAY_PARTITION	79	17.8%	split an array into independently addressable banks
DEPENDENCE	30	6.7%	assert that a dependence the tool sees is false
LOOP_TRIPCOUNT	12	2.7%	supply an expected bound for a data-dependent loop
RESOURCE	10	2.2%	choose the primitive that implements a storage or operator
STREAM	10	2.2%	set stream depth or implementation
BIND_STORAGE	8	1.8%	modern replacement for RESOURCE on memories
LOOP_MERGE	3	0.7%	fuse adjacent loops into one
LATENCY	2	0.4%	constrain a region's latency
DATAFLOW	2	0.4%	schedule regions concurrently
BIND_OP	2	0.4%	—
LOOP_FLATTEN	1	0.2%	—
PERFORMANCE	1	0.2%	—

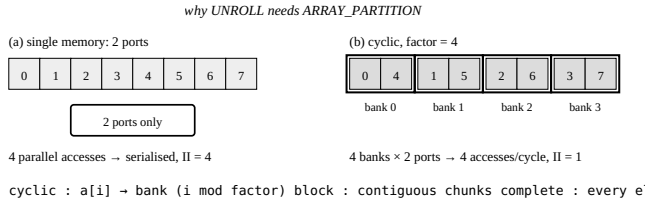


Fig. 14. Array partitioning. Unrolling is legal only when the memory can deliver the required number of accesses per cycle; cyclic partitioning is used when the access stride is one.

Table 3 reports the arbitrary-precision and stream types. The dominance of `ap_uint` over `ap_fixed` is a consequence of corpus composition: cryptographic and quantised-network code is integral, whereas only the solver family carries fractional data. The 250 uses of `hls::stream` are concentrated almost entirely in FINN and in the top-level solver entry points, and this concentration is itself one of the findings of Section VI-D.

Table 3. Arbitrary-precision and stream type usage.

Type	Uses	Role
<code>ap_uint<</code>	457	unsigned integer of arbitrary width
<code>hls::stream<</code>	250	blocking FIFO channel between stages
<code>std::complex<</code>	63	standard complex, supported for float paths
<code>hls::x_complex<</code>	62	synthesis-friendly complex number
<code>ap_fixed<</code>	35	signed fixed point, total and integer width
<code>ap_int<</code>	20	signed integer of arbitrary width
<code>decltype</code>	18	deduce accumulator type from the activation

III. THE LINEAR-ALGEBRA FAMILY

The Vitis solver family implements matrix decompositions that recur throughout wireless detection, radar beamforming, state estimation and computational finance. All four files share a structure: a traits template carrying internal types and architecture switches; two or three alternative implementations of the same mathematics; a dispatcher that selects among them; and a stream-interfaced top-level entry point. We treat `cholesky.hpp` in detail because it exhibits the pattern most clearly, then note what differs in the others.

A. Cholesky Decomposition — `cholesky.hpp` (732 lines)

1) Problem and contract

Given a Hermitian positive-definite matrix A , the decomposition produces a lower triangular L with $A = LL^H$. The public entry point is stream-interfaced and, critically, returns a status:

```
template <bool LowerTriangularL, int RowsColsA,
          class InputType, class OutputType,
          typename TRAITS =
choleskyTraits<LowerTriangularL,
               RowsColsA, InputType,
               OutputType> >
int cholesky(hls::stream<InputType>& matrixAStrm,
             hls::stream<OutputType>& matrixLStrm);
```

The return type is `int`, not `void`. If a diagonal entry becomes negative—which happens exactly when A is not positive definite—the core sets `return_code = 1` and the caller can act. This is a deliberate interface decision and it is, as Section VI-H argues, the single most commonly omitted feature in student and portfolio IP.

2) The traits structure

Parameterisation is carried by a traits template rather than by a long list of template arguments:

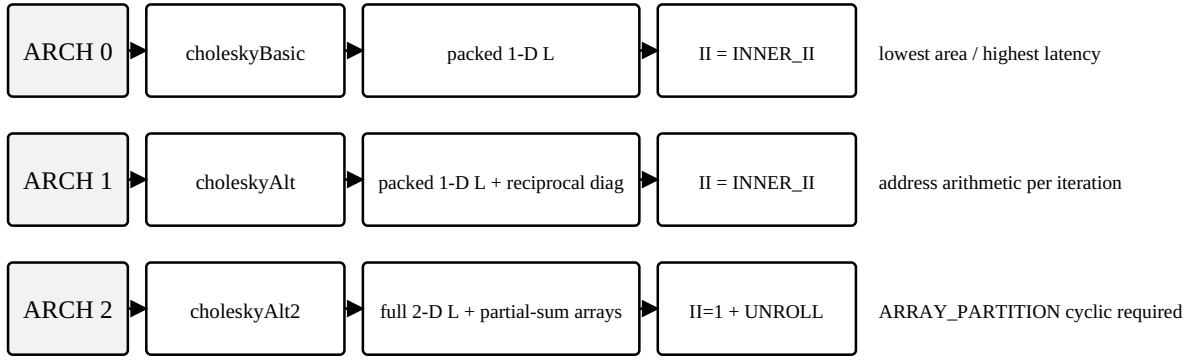
```

template <bool LowerTriangularL, int RowsColsA,
          typename InputType, typename OutputType>
struct choleskyTraits {
    typedef InputType  PROD_T;      // product
    typedef InputType  ACCUM_T;     // accumulator
    typedef InputType  ADD_T;       // subtract
    typedef InputType  DIAG_T;      // diagonal
    typedef InputType  RECIP_DIAG_T; // 1/diagonal
    typedef InputType  OFF_DIAG_T;  // off-diagonal
    typedef OutputType L_OUTPUT_T;  // stored result
    static const int ARCH = 1;
    static const int INNER_II = 1;
    static const int UNROLL_FACTOR = 1;
    static const int UNROLL_DIM = (LowerTriangularL ?
1 : 2);
    static const int ARCH2_ZERO_LOOP = true;
};

```

Two things are notable. First, *seven distinct internal types* are named. The width of a product is not the width of an accumulator, and neither is the width at which a reciprocal should be stored; conflating them either wastes area or overflows. Second, the same structure carries *architecture* switches. The consumer of this IP selects a circuit by choosing a number, and the vendor ships three circuits for one algorithm.

The template is specialised for `hls::x_complex`, `std::complex`, `ap_fixed`, and `complex` forms of `ap_fixed`. The specialisations differ in the widths they assign: complex accumulation needs additional headroom, and fixed-point reciprocals need more fractional bits than the value they invert.



area → increases downward, latency → decreases downward

Fig. 2. The three Cholesky architectures selected by `CholeskyTraits::ARCH`. Identical mathematics, three distinct circuits.

3) The three architectures

Fig. 2 summarises the selection. `choleskyBasic` is the reference: minimum resources, maximum latency. `choleskyAlt` reduces latency at the cost of resources and stores L packed into a one-dimensional array of $(N^2 - N)/2$ words, halving memory. The price is an address computation on every iteration:

```

int i_sub1 = i - 1;
int i_off = ((i_sub1 * i_sub1 - i_sub1) / 2) + i_sub1;
...
prod = -L_internal[i_off + k] *
hls::x_conj(L_internal[j_off + k]);

```

`choleskyAlt2` reverses that decision, and the source comment states the reason without ambiguity:

```

// To avoid array index calculations every iteration this
// architecture uses a simple 2D array rather than a
// optimized/packed triangular matrix.

```

The address arithmetic sits on the recurrence that determines the achievable initiation interval. Doubling the memory removes it. Fig. 3 contrasts the two layouts. This is the first of the general rules we extract: *when address generation limits the initiation interval, spend memory to delete it*.

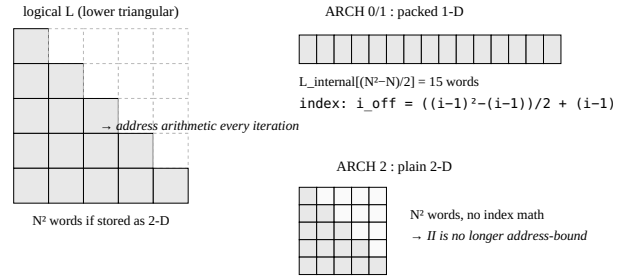


Fig. 3. Memory organisation trade-off in `cholesky.hpp`. ARCH 2 spends 2x memory to remove the per-iteration address computation that limits the initiation interval.

4) Elimination of division

Both improved architectures store the reciprocal of the diagonal rather than the diagonal itself. The comment is explicit: the intent is to avoid the latency of a divide in the inner loop. Every subsequent off-diagonal update is then a multiplication by `diag_internal[j]`. A dedicated helper, `cholesky_rsqrt`, is specialised for `ap_fixed` so that the reciprocal square root is computed once per column in the appropriate format.

The same move recurs throughout hardware arithmetic and it generalises beyond reciprocals: a normalisation by a constant factor $\alpha = 3/4$ is written as `(|v| * 3) >> 2` rather than as a

division, and a scaling by a power of two is a shift. Division is the one arithmetic primitive with no cheap hardware realisation, and production code arranges never to need it in a loop.

5) Partial sums and the necessity of banking

The highest-performance architecture carries two auxiliary arrays:

```
ACCUM_T square_sum_array[RowsColsA];
ACCUM_T product_sum_array[RowsColsA];

#pragma HLS ARRAY_PARTITION variable = square_sum_array
cyclic dim = 1 factor = UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable = product_sum_array
cyclic dim = 1 factor = UNROLL_FACTOR
#pragma HLS ARRAY_PARTITION variable = L_internal
cyclic dim = UNROLL_DIM factor = UNROLL_FACTOR
```

Rather than recomputing an inner product for each output element, the algorithm sweeps one column and updates the partial sums of *every* row simultaneously. Fig. 4 shows the dataflow. The unrolled inner loop therefore requires UNROLL_FACTOR concurrent accesses to each array, which a single memory with two ports cannot supply. Cyclic partitioning by the same factor creates exactly the banks required (Fig. 14).

The dependency runs in one direction only: the partition factor is derived from the unroll factor, not chosen independently. This is why ARRAY_PARTITION is the third most frequent directive in the corpus (Table 2) and why, in practice, an initiation interval that refuses to reach one is almost always a memory-port problem rather than an arithmetic one.

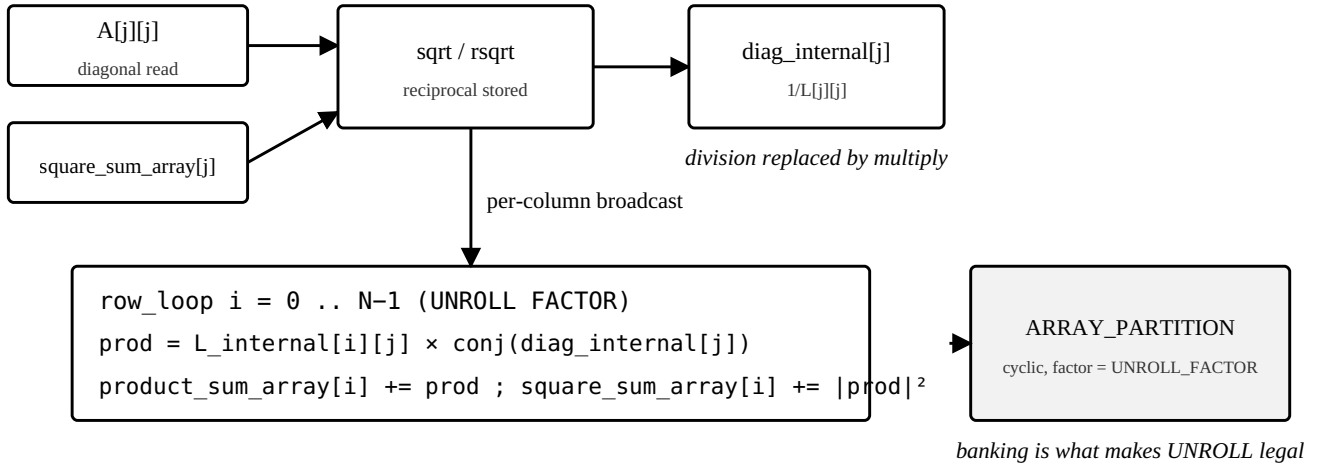


Fig. 4. choleskyAlt2 dataflow. Partial sums for every row are carried in arrays and updated during one column sweep; cyclic partitioning provides the simultaneous ports the unrolled loop needs.

6) Debug separation

Diagnostic output is present but excluded from synthesis:

```
if (cholesky_sqrt_op(A_minus_sum, new_L_diag)) {
#ifdef __SYNTHESIS__
    printf("ERROR: Trying to find the square root of a
    negative number\n");
#endif
    return_code = 1;
}
```

The pattern separates two audiences. During C simulation the message identifies which input broke the decomposition; after synthesis the same condition is still detected and still reported, but through the return code, which costs one flip-flop.

7) Function inventory

The file defines 9 functions and 19 template declarations. Besides the three architectures and the dispatcher, the helpers are: cholesky_rsqr (two overloads), cholesky_prod_sum_mult (three overloads, covering real, x_complex and std::complex mixed-type products), and cholesky_sqrt_op. The overload sets exist because hls::x_complex does not support multiplication by a real of a different type; rather than widening the operand, the library supplies the exact operation needed.

B. QR Factorisation — qrf.hpp (889 lines)

QR factorisation is implemented by Givens rotations rather than by Gram–Schmidt or Householder reflections. The choice is a hardware one: a Givens rotation touches exactly two rows and requires only a 2×2 product, so the computation decomposes into a regular array of identical small operations with short dependence chains. Modified Gram–Schmidt, by contrast, has a long serial reduction per column, and Householder needs a full-vector norm before any output is produced.

Givens rotation annihilates one sub-diagonal element

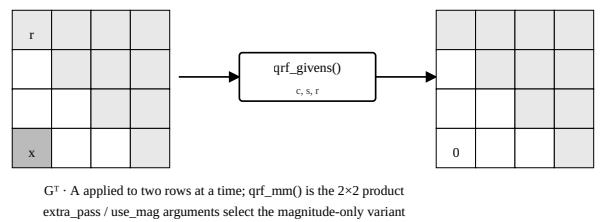


Fig. 5. Givens-rotation QR in qrf.hpp. Each rotation zeroes one element; qrf_mm applies the 2x2 rotation to a row pair.

The core helpers are `qrf_givens`, which computes the rotation coefficients c , s and the resulting r ; and `qrf_mm`, which applies a 2×2 rotation to a pair of values. A variant, `qrf_mm_or_mag`, takes `use_mag` and `extra_pass` flags and selects between applying the rotation and substituting a precomputed magnitude. The `extra_pass` mechanism exists to handle fixed-point cases where a single pass loses accuracy; the same function body serves both passes, parameterised by a flag, rather than being duplicated.

`qrf_givens` has three overloads covering the `real`, `x_complex` and `std::complex` cases. The file declares 22 templates in 889 lines—the highest template density in the solver family—because every arithmetic helper is specialised per input type rather than being written once against a generic numeric concept.

The basic architecture, `qrf_basic`, is stream-interfaced like Cholesky. As in `cholesky.hpp`, a traits structure selects the architecture and carries the internal types; 22 HLS pragmas appear in the file, with `PIPELINE` on the rotation-application loops and `ARRAY_PARTITION` on the working matrix.

C. Singular Value Decomposition — `svd.hpp` (1577 lines)

At 1577 lines this is the largest file in the solver family and the second largest in the corpus. It implements two-sided Jacobi SVD: repeated application of rotation pairs that annihilate off-diagonal elements, iterated in sweeps until the off-diagonal norm falls below a tolerance.

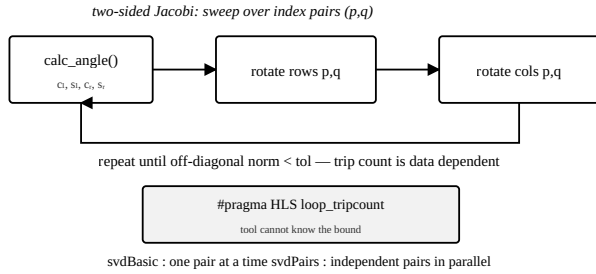


Fig. 6. Two-sided Jacobi SVD in `svd.hpp`. The iteration count depends on the data, which is why `loop_tripcount` appears; `svdPairs` exploits that disjoint index pairs commute.

The structure differs from Cholesky and QR in one respect that matters for HLS. The number of sweeps is *not known at compile*

time; it depends on the data. This is the canonical irregular case, and the file handles it the only way available:

```
#pragma HLS loop_tripcount max = ...
```

The directive does not constrain the hardware. It supplies the reporting tool with an expected bound so that latency estimates are meaningful. The hardware itself must terminate on a data-dependent condition, which means the block has variable latency and its consumer must be prepared for it. This is precisely the property that makes irregular algorithms expensive in HLS: the pipeline can still be filled, but the schedule cannot be static.

Two architectures are provided. `svdBasic` processes one index pair at a time. `svdPairs` exploits the fact that rotations acting on disjoint index pairs commute, and processes several concurrently; this is the classical parallel-ordering optimisation, and it is exposed to the IP consumer through the architecture switch in the traits structure. `calc_angle` computes the rotation angles and is specialised for the complex cases. The file carries 44 pragmas, the highest count in the solver family.

D. Blocked Cholesky — `potrf.hpp` (200 lines)

This file sits at what the Vitis Libraries call level 2: a composition layer above the level-1 kernels. Its entry point deliberately mirrors the LAPACK name and signature:

```
template <typename T, int NMAX, int NCU>
void potrf(int m, T* A, int lda, int& info);
```

Three observations. The matrix dimension m is a *runtime* argument while $NMAX$ is a compile-time bound—the hardware is built for the maximum and the runtime value controls how much of it is exercised. NCU , the number of computation units, is the folding parameter: rows are distributed across NCU engines, with the working array declared as `dataA[NCU][ (N+NCU-1)/NCU ] [N]` so that the unit index is the outermost dimension and each unit owns a private bank. And `info` is an output reference carrying the LAPACK failure convention, again making the failure path explicit.

The internal decomposition is into `chol_col`, `chol_jj`, `chol_col_wrapper` and `cholesky_core`. The separation between a *wrapper* that distributes work across units and a *core* that performs the factorisation is the level-1/level-2 boundary in miniature: the core is the reusable kernel, the wrapper is the integration decision. 24 pragmas appear in only 200 lines, the highest density in the solver family, because at this level almost every array must be explicitly banked across the NCU dimension.

Table 4. Solver family: structure of the four files.

File	Lines	Fn	Tmpl	Prag	Architectures provided
<code>cholesky.hpp</code>	732	9	19	17	Basic / Alt / Alt2
<code>qrf.hpp</code>	889	11	22	22	basic + magnitude/extra-pass variants
<code>svd.hpp</code>	1577	11	19	44	svdBasic / svdPairs
<code>potrf.hpp</code>	200	5	5	24	NCU-parallel blocked

IV. THE CRYPTOGRAPHIC FAMILY

The security family is the most regular code in the corpus. Block ciphers and hash functions have fixed round counts, no data-dependent branching and no dynamic memory behaviour; they are, in the terms of Section VI-F, the case HLS handles best. The value of reading them lies not in algorithmic subtlety but in seeing how

bit-level manipulation is expressed and how storage primitives are chosen.

A. Advanced Encryption Standard — `aes.hpp` (1010 lines)

1) Class organisation

The file defines 9 classes. The root, `aesTable`, holds the substitution tables. `aesEnc<W>` and `aesDec<W>` are primary templates whose bodies are empty; the work is done in explicit specialisations for $W \in \{128, 192, 256\}$. This is a deliberate choice over a runtime key-length parameter: each key length has a different round count and a different key schedule, and specialising eliminates both from the hardware.

2) Separation of key expansion from block processing

Each specialisation exposes two members:

```
void updateKey(ap_uint<W> cipherkey);
void process(ap_uint<128> plaintext,
            ap_uint<W> cipherkey,
            ap_uint<128>& ciphertext);
```

The expanded schedule lives in `key_list[]` between calls. A session that encrypts many blocks under one key runs the schedule once. Fig. 12 shows the split. The general principle—*identify state that outlives one datum and give it its own entry point*—recurs in FINN, where weights are loaded once and reused across NF folds, and in the solver family, where a factorisation is amortised over many right-hand sides.

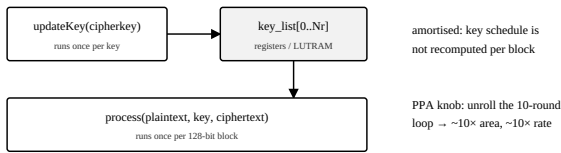


Fig. 12. State-reuse partitioning in `aes.hpp`: key expansion and block processing are separate entry points so the schedule is computed only when the key changes.

3) The round function

The round body is worth quoting because it shows how bit manipulation maps to hardware:

```
state = plaintext ^ key_list[0];
for (round_counter = 1; round_counter <= 10;
    round_counter++) {
    for (int i = 0; i < 16; i++) {
        #pragma HLS unroll
        state(i*8+7, i*8) = ssbox[state(i*8+7, i*8)]; //
    }
    SubBytes
    tmp_1 = state(15, 8);
    state(15, 8) = state(47, 40); //
    ShiftRows
    state(47, 40) = state(79, 72);
    ...
    if (round_counter < 10) { /* MixColumns: GFmul2,
        GFmul3 */
        state ^= key_list[round_counter];
    }
}
```

`ShiftRows` consists entirely of bit-slice assignments on an `ap_uint<128>`. In software this is data movement; in hardware it is *rewiring* and costs nothing. The permutation is free, and recognising which operations are free is a large part of reading HLS code correctly. Fig. 11 shows the round datapath with this annotation.

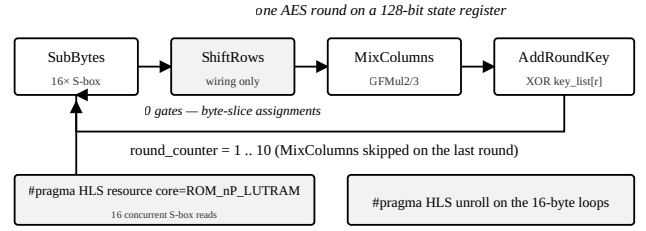


Fig. 11. AES round datapath in `aes.hpp`. `ShiftRows` costs no logic because `ap_uint<128>` bit-slice assignment is pure rewiring.

4) Storage selection

The substitution tables carry an explicit resource directive:

```
#pragma HLS resource variable = iibox core = ROM_nP_LUTRAM
```

The choice of LUT-based ROM over block RAM is forced by the access pattern. `SubBytes` is unrolled over all sixteen bytes, so sixteen independent lookups must complete in one cycle. A block RAM offers two ports; distributed LUT ROM can be replicated as many times as the unrolled loop requires. The pragma is not a hint here, it is a correctness-of-schedule requirement, and it illustrates the same dependency seen in the solver family: unrolling is only legal when the memory can serve it.

5) Galois-field arithmetic

`MixColumns` requires multiplication in $GF(2^8)$ by the constants 2 and 3. The file provides `GFmul2` and `GFmul3` as helper functions rather than using a general multiplier. Multiplication by 2 is a shift and a conditional exclusive-or with the field polynomial; multiplication by 3 is that result exclusive-ored with the operand. Both reduce to a handful of gates. The decryption path additionally requires multiplication by 9, 11, 13 and 14, which is why `aesDec` is larger than `aesEnc` and why inverse `MixColumns` dominates the decryption area.

6) The single PPA control

Unlike the solver family, this file exposes no architecture switch. The resource–throughput trade is controlled by whether the round loop is unrolled: rolled, the core occupies one round of logic and takes ten cycles per block; fully unrolled, it becomes a ten-stage pipeline with roughly ten times the area and ten times the throughput. Partial unrolling gives intermediate points. This one-dimensional design space is characteristic of fixed-round cryptographic primitives.

B. SHA-224/256 — `sha224_256.hpp` (854 lines)

The hash core is organised as a small dataflow pipeline rather than as a single function: message padding, message-schedule generation and the round loop are separate stages connected by streams. Fig. 13 shows the arrangement. Because the schedule word W_t for round t depends only on earlier schedule words, the schedule stage can run ahead of the compression stage, and the two overlap.

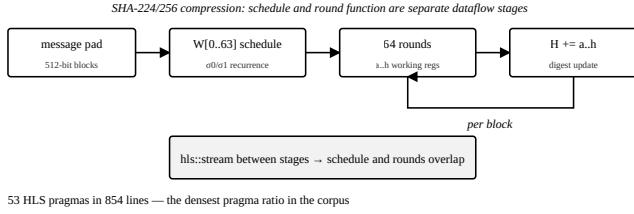


Fig. 13. Structure of sha224_256.hpp. Message expansion and the round loop are separate stream-connected stages so that they execute concurrently.

The file carries 53 pragmas in 854 lines—a density of 6.2 per hundred lines, the highest in the corpus. Two features account for this. The round loop is fully unrolled over the eight working variables, each of which becomes a register, and the rotation and shift operations on 32-bit words become fixed wiring. And the file supports both SHA-224 and SHA-256 from one body, differing

only in the initial hash value and the truncation of the output, so the conditional paths need explicit scheduling control.

Debug support follows the pattern seen in `cholesky.hpp`: a `_XF_SECURITY_PRINT` macro is defined to nothing unless `_DEBUG` is set, and `<cstdio>` is included only when `__SYNTHESIS__` is undefined.

C. Shared Types and Utilities

Two small files complete the family. `xf_security/types.hpp` (58 lines) defines packed structures used as stream payloads—typically a data word paired with an end-of-message flag—so that control travels with data through the stream network rather than on side channels. `xf_security/utils.hpp` (44 lines) provides endianness conversion and word-assembly helpers. Neither contains a pragma, and both are pure type definitions; they are included here for completeness of coverage.

Table 5. Cryptographic family: structure of the four files.

File	Lines	Fn	Cls	Prag	Standard
<code>aes.hpp</code>	1010	8	9	45	FIPS 197, key 128/192/256
<code>sha224_256.hpp</code>	854	16	4	53	FIPS 180-4
<code>xf_security/types.hpp</code>	58	0	0	0	stream payload types
<code>xf_security/utils.hpp</code>	44	0	0	0	endianness and word assembly

V. THE QUANTISED NEURAL-NETWORK FAMILY

FINN-hlslib is the largest and most fragmented part of the corpus: 16 files and 6,528 lines. The fragmentation is structural rather than

incidental. An accelerator is assembled by instantiating a chain of stream-connected stages, each of which is independently folded, and each stage is a separate header. Fig. 10 shows a representative chain.

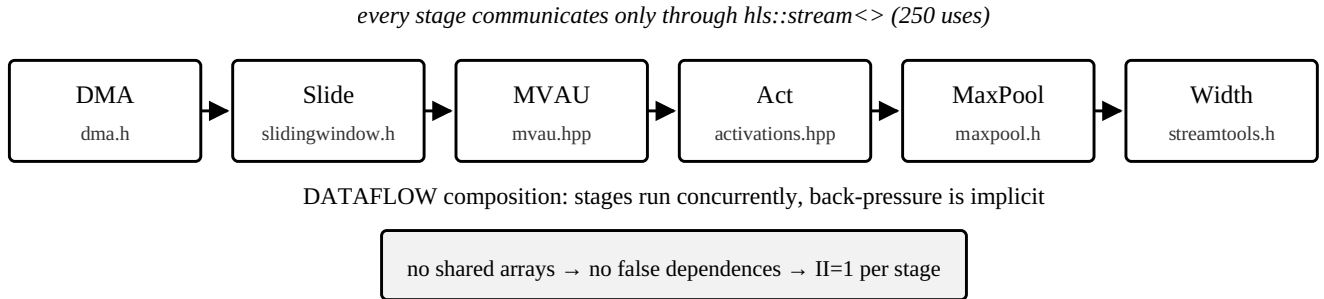


Fig. 10. FINN composes an accelerator as a chain of stream-connected stages; the stream abstraction is what makes concurrent scheduling and back-pressure automatic.

A. Matrix–Vector–Activate Unit — `mvau.hpp` (310 lines)

1) Folding

This is the computational core of the library and the clearest statement of the folding idea in the corpus:

```
template<unsigned MatrixW, unsigned MatrixH,
        unsigned SIMD, unsigned PE, unsigned MMV, ... >
void Matrix_Vector_Activate_Batch(hls::stream<TI> &in,
                                   hls::stream<TO> &out,
                                   TW const &weights,
                                   TA const &activation,
                                   int const reps, R const
&r) {
    unsigned const NF = MatrixH / PE;    // vertical folds
    unsigned const SF = MatrixW / SIMD;  // horizontal
    folds
    unsigned const TOTAL_FOLD = NF * SF;
```

The physical arithmetic is SIMD×PE multiply–accumulate units, fixed at compile time. A larger matrix does not produce a

larger circuit; it produces more cycles. Fig. 7 shows the tiling. Three parameters—SIMD, PE and MMV—span the entire resource-throughput design space of the layer, and the same two divisions determine both the cycle count and the buffer depths.

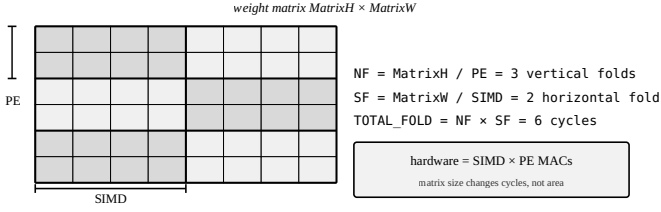


Fig. 7. Folding in mvau.hpp. Physical MAC resources are fixed at SIMD x PE; a larger matrix costs more cycles, never more area.

2) Loop flattening

The nested fold loops are merged into a single iteration space, and the source comment states why:

```
// everything merged into a common iteration space (one
"big"
// loop instead of smaller nested loops) to get the
// pipelining the way we want
unsigned const TOTAL_FOLD = NF * SF;
for(unsigned i = 0; i < reps * TOTAL_FOLD; i++) {
#pragma HLS pipeline style=flp II=1
    ...
    if(++sf == SF) { sf = 0; ... ++nf; }
}
```

A nested loop pipeline drains at every outer-loop boundary: the inner pipeline empties before the outer iteration advances, and the cost is proportional to the pipeline depth times the number of outer iterations. Flattening removes the boundary entirely and the counters *sf*, *nf* and *tile* are maintained by hand. Fig. 8 illustrates the difference. This is among the most transferable techniques in the corpus, because the same reasoning applies verbatim to a hand-written RTL controller.

(a) nested loops — pipeline drains at every outer boundary



(b) merged iteration space — II = 1 throughout

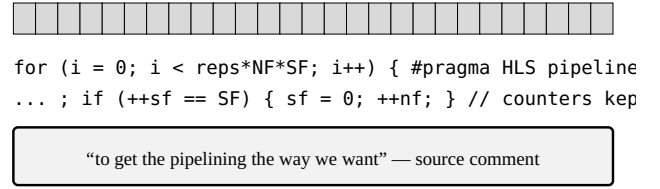


Fig. 8. Loop flattening. FINN merges the nested fold loops into one iteration space and maintains *sf/nf/tile* manually so the pipeline never drains.

3) Input reuse and accumulator placement

Two further decisions are visible in the same loop body:

```
if(nf == 0) { inElem = in.read(); inputBuf[sf] = inElem; }
else { inElem = inputBuf[sf]; }
...
decltype(activation.init(0,0)) accu[MMV][PE];
#pragma HLS ARRAY_PARTITION variable=accu complete dim=0
```

The input vector is read from the stream exactly once and buffered for reuse across the *NF* vertical folds; the buffer depth is *SF*, not the matrix width. And the accumulator array is partitioned *completely*, which turns every element into a register. Complete partitioning is appropriate here because the array is small and every element is written every cycle; cyclic partitioning, as used in the solver family, is the right choice when the array is large and the access stride is one.

The accumulator type is deduced with `decltype(activation.init(0,0))` rather than being named. The activation object therefore determines the accumulator width, which keeps the numeric contract in one place instead of two.

B. Vector–Vector–Activate — vvau.hpp (276 lines)

The depthwise counterpart of the MVAU. In a depthwise convolution each output channel depends on exactly one input channel, so the weight matrix is block diagonal and the SIMD dimension collapses. The file is 276 lines with 14 pragmas and mirrors the MVAU structure—the same flattened loop, the same folding counters—with the inner product taken over the kernel window rather than over input channels.

C. Sliding Window Generator — slidingwindow.h (2095 lines)

At 2,095 lines this is the largest file in the entire corpus, larger than any solver, and it computes no arithmetic at all. Its task is to convert a raster-order pixel stream into the sequence of overlapping $K \times K$ windows that the MVAU consumes: the hardware equivalent of `im2col`. Fig. 9 shows the line-buffer structure.

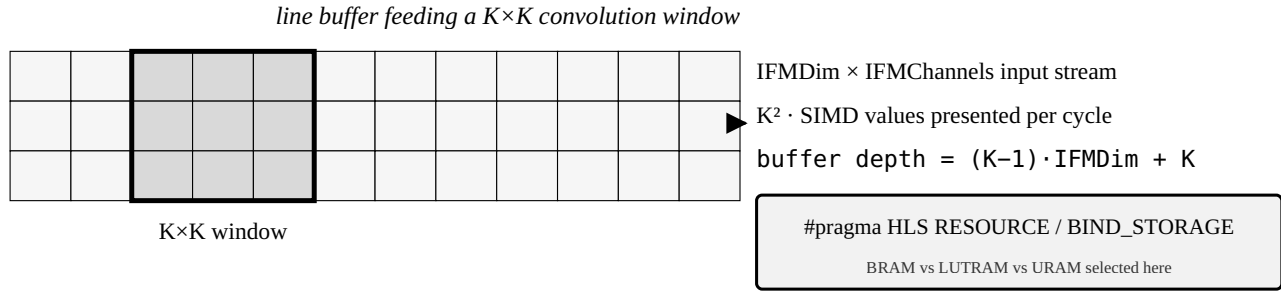


Fig. 9. slidingwindow.h (2,095 lines, the largest file). A line buffer converts a raster stream into overlapping windows; the storage pragma chooses the memory primitive.

The size is explained by the number of cases. Stride one and stride greater than one need different buffer management; padding must be synthesised at the borders; the input may be narrower or wider than the buffer word; depthwise and standard convolutions traverse channels differently; and the buffer itself may be mapped to block RAM, LUT RAM or URAM depending on its size. The file carries 66 pragmas, second only to slidingwindow's own size in the corpus, and 19 distinct function definitions covering these combinations.

That a data-movement block is the largest component is itself a finding. In quantised inference the multiply–accumulate array is cheap; delivering operands to it at rate is expensive. The same asymmetry appears in the solver family, where memory banking rather than arithmetic determines the achievable initiation interval.

D. Stream Utilities — *streamtools.h* (1008 lines)

23 functions in 1,008 lines, all of signature stream-to-stream. The operations are: width conversion in both directions, padding and cropping of feature maps, duplication of a stream to several consumers, limiting the number of packets, insertion of explicit FIFOs, flattening and lane interleaving. Fig. 16 illustrates width conversion.

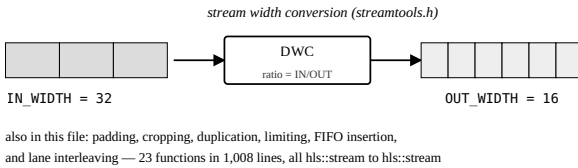


Fig. 16. Stream utilities. Width conversion decouples the natural word size of neighbouring stages so that each can be folded independently.

Width conversion deserves comment because it is what makes independent folding possible. If stage A naturally produces SIMD_A values per cycle and stage B consumes SIMD_B, a converter between them lets each stage be folded for its own resource budget without forcing a common word size on the whole chain. Without it the folding factors of adjacent stages would be coupled, and the design space would collapse to a single global parameter.

E. Pooling — *maxpool.h* (609 lines)

12 functions and 35 pragmas. Max pooling over binary values degenerates to a logical OR, and the file provides that specialisation separately from the general comparison tree. The general case reuses the sliding-window machinery to present

pooling windows, then reduces each window with a comparator tree whose depth is logarithmic in the window size. Average pooling is also present and is more expensive: it requires an adder tree and a division by the window area, which is implemented as a multiplication by a precomputed reciprocal—the same substitution seen in *cholesky.hpp*.

F. Activations and Thresholding — *activations.hpp* (387 lines)

19 class definitions in 387 lines. The central idea is that for a quantised network the activation function and the re-quantisation step collapse into a single comparison against a monotone threshold set. Instead of scaling an accumulator and applying a nonlinearity, the hardware compares the accumulator against $T_0 < T_1 < \dots$ and emits the index of the interval. Fig. 17 shows the structure.

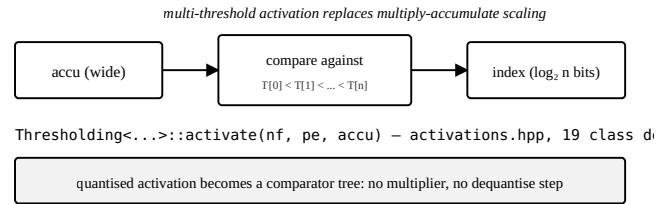


Fig. 17. Threshold-based activation in activations.hpp. For quantised networks the activation and re-quantisation collapse into a single comparator tree.

The consequence is that no multiplier appears in the activation path at all. The classes provided include Identity, binary and multi-bit thresholding with per-channel parameters, and variants that fold a bias into the threshold set. The `activation.init(nf, pe)` member seen in the MVAU belongs to this interface: it supplies the initial accumulator value, which is how a bias is added without an extra adder.

G. Weight Storage — *weights.hpp* (169 lines)

5 classes in 169 lines. The file adapts the logical weight matrix to the physical organisation the folded datapath needs. The MVAU requests `weights.tile()` and expects PE rows of SIMD values; the storage class is responsible for having arranged the memory so that this is a single access. For binary networks the packing is dense—SIMD weights occupy SIMD bits—and the file provides a specialised adapter for that case. The 9 pragmas are all storage-binding directives.

H. Type Interpretation — *interpret.hpp* (307 lines)

11 classes and 36 pragmas in 307 lines. This file answers a question that does not arise in floating-point code: given a packed bit vector, what numbers does it represent? The interpreters cover unsigned and signed binary, ternary, and fixed-point encodings, and they are supplied to the MVAU as the `TSrcI`, `TDstI` and `TweightI` template parameters. Separating interpretation from storage means that the same packed memory can be read as signed or unsigned without copying, and that the multiply-accumulate primitive can specialise on the pair of interpretations it is given.

I. Multiply-Accumulate Primitives — *mac.hpp* (206 lines)

The lowest level of the library. The header comment states the intent: perform the product of two operands and let HLS choose the best resource. For binary operands the product is an exclusive-nor and the accumulation is a population count, so no DSP block is used at all. For higher precisions the operation maps to a DSP slice. The file carries 10 pragmas in 206 lines and exists so that the MVAU body can be written once against an abstract `mac<SIMD>` regardless of the precision in use.

J. Direct Memory Access — *dma.h* (238 lines)

The boundary between the accelerator and the memory system. The header comment describes it as a block accessing AXI4 memory and producing HLS streams. Two directions are provided, memory-to-stream and stream-to-memory, with the burst length and the AXI word width as template parameters. 6 functions in 238 lines. This is where the `INTERFACE` pragmas that define the external protocol appear, and it is the only part of the library that knows about addresses; every other stage sees streams only.

K. Layer Composition — *convlayer.h* (311 lines)

12 functions in 311 lines with only 3 pragmas. The file contains almost no logic: it instantiates a sliding-window generator, an MVAU and the required stream adapters, and wires them together. The low pragma count is the point—composition is structural, and

the scheduling decisions have already been made inside the components. This is the level at which an IP consumer works, and the parameter list of the layer function is the union of the folding parameters of its parts.

L. Fault Tolerance — *tmrcheck.hpp* (204 lines)

Triple modular redundancy: three replicas of a computation, a majority vote, and an error flag raised when the replicas disagree. Fig. 18 shows the structure. Its presence in a neural-network library is worth noting. It is the only component in the corpus that addresses reliability rather than performance, and it indicates that the library targets deployments—automotive, aerospace—where a functional-safety argument is required. The cost is a factor of three in area, which is why it is an opt-in component rather than a property of the datapath.

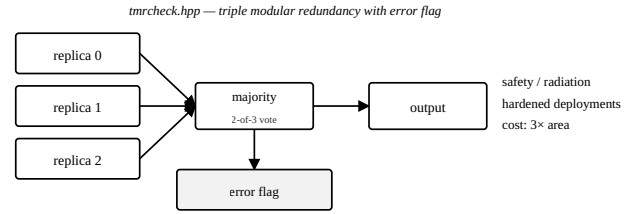


Fig. 18. Triple modular redundancy in *tmrcheck.hpp*: the corpus ships a fault-tolerance option as an ordinary library component.

M. Remaining Components

upsample.hpp (182 lines) implements nearest-neighbour upsampling for square feature maps, used in decoder paths. *mmv.hpp* (59 lines) defines the multiple-matrix-vector type that carries several pixels through one datapath. *utils.hpp* (107 lines) provides a non-synthesisable stream logger for debugging, guarded so that it disappears in hardware. *bnn-library.h* (60 lines) is a convenience header that includes the others and contains no definitions of its own. All four are listed in full in Appendix A.

Table 6. FINN-hlslib: all sixteen files.

File	Lines	Fn	Cls	Prag	Role
<i>slidingwindow.h</i>	2095	19	0	66	raster stream to convolution windows
<i>streamtools.h</i>	1008	23	4	24	width conversion, padding, duplication, FIFO
<i>maxpool.h</i>	609	12	0	35	max and average pooling
<i>activations.hpp</i>	387	14	19	15	threshold-based quantised activation
<i>convlayer.h</i>	311	12	0	3	composition of a convolution layer
<i>mvau.hpp</i>	310	2	0	16	matrix-vector-activate, the compute core
<i>interpret.hpp</i>	307	13	11	36	bit-vector to number interpretation
<i>vvau.hpp</i>	276	2	0	14	depthwise vector-vector-activate
<i>dma.h</i>	238	6	0	3	AXI4 memory to stream and back
<i>mac.hpp</i>	206	2	0	10	multiply-accumulate primitives
<i>tmrcheck.hpp</i>	204	2	0	6	triple modular redundancy
<i>upsample.hpp</i>	182	3	0	2	nearest-neighbour upsampling
<i>weights.hpp</i>	169	3	5	9	packed weight storage adapter
<i>utils.hpp</i>	107	3	8	0	non-synthesisable stream logger
<i>mmv.hpp</i>	59	0	1	1	multiple-matrix-vector payload type
<i>bnn-library.h</i>	60	0	0	0	aggregate include header

VI. CROSS-CUTTING ANALYSIS

Having read the three families separately, we now ask what they share. The families were written by different teams for unrelated domains under different licences, so agreement between them is evidence about HLS practice rather than about one project's conventions.

A. Pragma Usage as a Map of the Difficulty

Table 2 gave the raw distribution. The interpretation is more useful than the numbers. Directives fall into three groups by what they tell the tool.

Scheduling directives—PIPELINE, UNROLL, INLINE, LOOP_MERGE—state the designer's throughput intent. They express what the circuit should do. *Structural directives*—ARRAY_PARTITION, RESOURCE, BIND_STORAGE, STREAM—make

that intent physically possible by supplying ports and choosing primitives. *Assertions*—DEPENDENCE, LOOP_TRIPCOUNT—supply knowledge the compiler cannot derive.

The ratio between the first two groups is the practical lesson. Scheduling directives outnumber structural ones, but a scheduling directive without its matching structural directive does not fail loudly; it silently produces a worse initiation interval. The corpus never unrolls an array-touching loop without partitioning the array, and the partition factor is always derived from the unroll factor rather than chosen independently.

The assertion group is small but marks the hardest places. DEPENDENCE is an unchecked promise: if the designer is wrong about a dependence being false, the hardware is wrong and simulation of the C model will not reveal it. LOOP_TRIPCOUNT marks data-dependent iteration and appears predominantly in `svd.hpp`, the one genuinely irregular algorithm in the corpus.

the pattern common to all three families

1	parameterise	template args or a Traits struct carrying internal types AND architecture switches
2	separate state	one entry point that prepares reusable state, another that streams data
3	fold	fixed hardware = $P \times Q$; problem size maps to cycles, not area
4	flatten	merge nested loops into one iteration space so the pipeline never drains
5	bank	partition every array the unrolled loop touches, cyclically along the access stride
6	remove division	store reciprocals; replace $/$ by $\times$ and $>>$
7	report failure	return a status code the caller can act on
8	separate debug	guard printf and assertions with <code>#ifndef __SYNTHESIS__</code>

Fig. 19. The eight recurring design moves observed in every family of the corpus, independent of application domain.

B. Parameterisation: Traits Versus Template Arguments

Two strategies appear. The solver family uses a *traits structure*: a small number of template arguments select the traits type, and the traits type carries both the internal numeric types and the architecture switches. The FINN family uses *direct template arguments*: SIMD, PE, MMV and the interpretation types appear in the function signature.

The difference is not stylistic. A traits structure is preferable when the parameters are numerous, interdependent and mostly defaulted—the consumer overrides one field and inherits the rest, and specialisations can adjust widths coherently for complex or fixed-point inputs. Direct arguments are preferable when the parameters are few, orthogonal and always supplied. Cholesky has seven internal types that must vary together with the input type; enumerating them at every call site would be unusable. The MVAU has three folding parameters that the caller must always choose deliberately; hiding them in a traits structure would obscure the only decision that matters.

Both are strictly better than the third option, which the corpus never uses: runtime parameters. Every structural parameter in all 25 files is a compile-time constant, because a runtime-variable fold factor would require the hardware to be built for the

maximum anyway. The one runtime dimension in the corpus is `potrf`'s matrix order m , and there the compile-time `NMAX` still sizes the circuit.

C. Memory Architecture

Three partitioning modes appear and the choice among them is determined by the access pattern, not by preference.

- **complete**—every element becomes a register. Used for the MVAU accumulator array, which is small and entirely rewritten each cycle.
- **cyclic**—element i goes to bank $i \bmod f$. Used for the Cholesky partial-sum arrays, where consecutive elements are consumed in the same cycle by an unrolled loop of stride one.
- **block**—contiguous ranges per bank. Appropriate when different units own disjoint regions, as in `potrf`, where the working array is declared `dataA[NCU][...][N]` so that the unit index is already the outermost dimension and no partitioning directive is needed at all.

The last case is worth emphasising because it is the cheapest technique in the corpus: *choosing the array's dimension order so that the parallel index is outermost makes banking implicit*. No pragma is required when the declaration already expresses the intent.

D. Interface Style

Table 7 shows where `hls::stream` is used. The concentration is extreme: FINN uses streams for every inter-stage connection, and the Vitis solvers use them only at the top-level entry points while working on arrays internally.

Table 7. Concentration of stream interfaces (top eight files).

File	<code>hls::stream</code> uses	Interface style
<code>streamtools.h</code>	63	streaming throughout
<code>sha224_256.hpp</code>	51	streamed boundary, array interior
<code>slidingwindow.h</code>	34	streaming throughout
<code>convlayer.h</code>	24	streaming throughout
<code>maxpool.h</code>	24	streaming throughout
<code>qrf.hpp</code>	11	streamed boundary, array interior
<code>dma.h</code>	10	streaming throughout
<code>upsample.hpp</code>	6	streaming throughout

The consequence is architectural. Because FINN stages communicate only through streams, they share no arrays, so no false dependences can arise between them and each may be scheduled concurrently with an independent initiation interval. Back-pressure is implicit in the blocking read. The cost is that every stage must be written in a streaming style, and that a stage which genuinely needs random access to a large buffer—such as the sliding-window generator—must manage that buffer internally and present a stream interface outward. That is exactly why `slidingwindow.h` is the largest file in the corpus.

The solver family makes the opposite trade. A matrix decomposition needs random access to the working matrix by nature, so the internal representation is an array and only the boundary is streamed. The lesson for a designer is that the interface style should follow the access pattern of the algorithm, and that a streaming interface at the boundary can be provided regardless of what happens inside.

E. Folding and the Resource–Throughput Trade

Every core in the corpus exposes a resource–throughput control, and in every case it takes the same form: a compile-time divisor that maps problem size onto cycles while holding the arithmetic area fixed.

Table 8. The folding parameter in each core.

Core	Fold parameter	Fixed hardware	Cycles
cholesky ARCH 2	UNROLL_FACTOR	$O(P)$ multiply-accumulate	$O(N^3/P)$
potrf	NCU	NCU column engines	$O(N^3/NCU)$
svdPairs	concurrent index pairs	rotation units	sweeps $\times N/2P$
MVAU	$SIMD \times PE$	$SIMD \times PE$ MACs	$NF \times SF$
VVAU	PE	PE MACs	kernel $\times NF$
AES	round-loop unroll	1 to 10 round stages	10 down to 1

The names differ—UNROLL_FACTOR in Cholesky, NCU in `potrf`, SIMD and PE in the MVAU, round-loop unrolling in AES—but the structure is identical. Given a problem of size N and a fold factor P , the circuit contains $O(P)$ arithmetic and executes in $O(N/P)$ cycles. What differs between the families is only how many independent fold dimensions exist: one for AES, one for `potrf`, two for the MVAU (SIMD across inputs and PE across outputs), and three if MMV is counted.

This observation generalises to hand-written RTL without modification. A sub-rate parallel datapath in a serialiser–deserialiser receiver, where P lanes each run at $1/P$ of the symbol rate, is the same construction with the same trade.

F. Loop Structure and the Initiation Interval

Three techniques recur.

Flattening. The MVAU merges nested fold loops into one iteration space and maintains the counters by hand, because a nested pipeline drains at each outer boundary (Fig. 8). The cost of the drain is the pipeline depth times the number of outer iterations, which for a deeply folded layer dominates.

Recurrence shortening. The Cholesky ARCH 2 decision—replacing packed indexing with a plain two-dimensional array—is a recurrence-shortening move. The address computation sat on the loop-carried path; removing it shortens the recurrence and allows a lower initiation interval.

Partial-sum carrying. Rather than recomputing an inner product per output, both Cholesky ARCH 2 and the MVAU carry

partial sums in arrays and update them all in one sweep. This converts a reduction, which has a long dependence chain, into an accumulation across independent banks.

All three are RTL techniques stated in C. None of them is about the C++ language.

G. Numeric Type Management

The corpus never uses one width for a whole datapath. Cholesky names seven internal types; the MVAU deduces the accumulator type from the activation object with `decltype`; FINN separates *storage* from *interpretation* so that a packed memory can be read as signed or unsigned without copying.

The underlying rule is that the width of a product, the width of an accumulator and the width of a stored result are three different quantities. A product of two w -bit operands needs $2w$ bits; an accumulation of n such products needs $2w + \lceil \log_2 n \rceil$; the stored result may be narrower than either. Collapsing them to one parameter either wastes area at the product or overflows at the accumulator, and the failure mode of the latter is silent.

H. Failure Reporting and Debug Separation

Two conventions appear in every family. Failures are reported through a return value or an output reference: `cholesky` returns `int`, `potrf` writes `info` following the LAPACK convention, and the FINN thresholding classes saturate rather than wrap. Diagnostic output is present but excluded from synthesis by

`#ifndef __SYNTHESIS__` or by a macro that expands to nothing unless a debug symbol is defined.

The combination matters because the two audiences are different. During C simulation the designer wants a message identifying the offending input. In hardware the consumer wants a status bit it can branch on. Providing only the first leaves the deployed circuit silent; the corpus provides both and pays one flip-flop for it.

VII. EXTRACTED DESIGN RULES

Fig. 19 collects the eight moves that appear in all three families. We restate them here as rules, with the evidence for each and an assessment of whether the rule is specific to HLS or transfers to hand-written RTL.

Table 9. The eight rules, their evidence, and whether they transfer to hand-written RTL.

#	Rule	Evidence in the corpus	Transfers to RTL
1	Parameterise architecture, not just size	<code>choleskyTraits::ARCH</code> selects three circuits; <code>svdBasic</code> vs <code>svdPairs</code>	Yes — a generator parameter
2	Give long-lived state its own entry point	<code>aesEnc::updateKey</code> vs <code>process</code> ; MVAU weight reuse across <i>NF</i>	Yes
3	Fold: fixed arithmetic, size maps to cycles	<code>SIMD×PE</code> ; <code>NCU</code> ; <code>UNROLL_FACTOR</code>	Yes — sub-rate parallelism
4	Flatten nested loops to hold $II = 1$	MVAU merged iteration space with manual <code>sf/nf</code> counters	Partly — becomes FSM design
5	Bank every array an unrolled loop touches	<code>ARRAY_PARTITION</code> cyclic factor = <code>UNROLL_FACTOR</code> in <code>choleskyAlt2</code>	Yes — explicit memory instances
6	Remove division; store reciprocals	<code>diag_internal</code> holds $1/L_{jj}$; pooling multiplies by $1/\text{area}$	Yes
7	Report failure through a status	<code>cholesky</code> returns <code>int</code> ; <code>potrf</code> writes <code>info</code>	Yes — a status port
8	Separate debug from synthesis	<code>#ifndef __SYNTHESIS__</code> ; <code>_XF_SECURITY_PRINT</code>	Yes — <code>`ifndef SYNTHESIS</code>

Seven of the eight transfer unchanged. Only rule 4, loop flattening, is phrased in a way specific to HLS—but its content, that a controller should not idle at a loop boundary, is an ordinary finite-state-machine concern. The conclusion we draw is that the transferable content of this corpus is architectural, and that a designer who internalises these eight moves has acquired something largely independent of whether the eventual output is produced by a synthesis tool or written by hand.

VIII. THREATS TO VALIDITY

We state the limits of this study explicitly, because a survey that does not is inviting its reader to over-generalise.

The corpus compiles; it was not synthesised. After the header closure described in Section II-A was resolved, 25 of the 26 corpus files compile under `g++ -std=c++14` against the public Xilinx headers, and a small program exercising `ap_uint`, `ap_int`, `ap_fixed` and `hls::stream` builds and runs. This establishes that the sources are well formed and that the types behave as the text assumes. It establishes nothing about hardware. No file was put through a synthesis tool, because none is available in our environment: there is no Vitis HLS, no Catapult, and no open-source C-to-RTL compiler that accepts this dialect. Consequently every statement in Sections III–V about scheduling, initiation interval, area or the circuit a pragma induces remains an *inference from source text and from the stated intent in comments*. We have measured no latency, area or frequency figure for this corpus and report none.

The compilation is with a different compiler. Vitis HLS uses a Clang-derived front end with its own extensions; we used GCC with `-fpermissive`, which was required because one Xilinx macro, `_AP_UNUSED_PARAM`, is rejected by GCC's stricter template name-lookup rules. A file that compiles under GCC

therefore does not prove it compiles under Vitis HLS, although the converse failure would have been strong evidence of a problem.

The sample is small and vendor-biased. Three families, two vendors, both from the same FPGA ecosystem. Patterns common to all three may reflect a shared house style or shared tool constraints rather than a general truth about HLS. An application-specific-integrated-circuit HLS flow using a different tool might differ in ways this corpus cannot reveal.

Function extraction is regular-expression based. The inventory in the appendices was produced by pattern matching, not by parsing C++. Macro-generated definitions and unusual declarator syntax may be missed or miscounted. The counts should be read as close approximations with a systematic bias toward undercounting, not as exact figures from a compiler front end.

Reading is interpretation. Where a comment states intent we quote it. Where no comment exists, the architectural reading in Sections III through V is our inference, and a reader with access to the original designers might learn that a decision was made for a reason we did not identify.

The corpus is a snapshot. Files were retrieved on a single date from the default branch of each repository. These are living projects; the line counts will change.

IX. CONCLUSION

We surveyed 11,917 lines of production high-level-synthesis C++ across three independently developed IP families: matrix decompositions, cryptographic primitives, and quantised neural-network operators. Every file was inventoried mechanically and read in full.

The principal finding is that the transferable content of this code is not the C++ and not the pragmas considered individually, but a small set of architectural moves that the three families share despite having nothing else in common. A core is parameterised so that architecture is a consumer choice; state that outlives one

datum gets its own entry point; problem size maps to cycles through a fold factor while arithmetic area stays fixed; loops are flattened so that the pipeline does not drain; every array an unrolled loop touches is banked, with the factor derived from the unroll factor; division is removed by storing reciprocals; failure is reported through a status the caller can act on; and debug code is separated by `__SYNTHESIS__`.

Seven of these eight apply without modification to hand-written register-transfer-level design. This is, we think, the most useful thing the corpus has to say to a designer deciding how much to invest in HLS: the tool is a means of expressing these decisions, not a substitute for making them. The decisions are the skill.

The study's principal limitation is that it is a reading, not a measurement. We could not compile the corpus, and we report no performance figures for it. A natural continuation would be to reproduce two or three of these cores in an open toolchain, synthesise them, and test whether the architectural choices the source makes are the ones a measurement would endorse.

APPENDIX A COMPLETE FILE INVENTORY

Every file in the corpus, with the metadata extracted mechanically as described in Section II-C. Lines and bytes are exact; function, template and class counts are pattern-matched and carry the caveat stated in Section VIII.

Table A1. Complete file inventory.

File	Lines	Bytes	Fn	Tmpl	Cls	Def	Lbl	Prag	Licence
Vitis_dsp_fft/vt_fft.hpp	25	856	0	0	0	1	0	0	Apache-2.0
Vitis_security/aes.hpp	1,010	45,200	8	8	9	1	1	45	unknown
Vitis_security/sha224_256.hpp	854	35,288	16	7	4	13	0	53	Apache-2.0
Vitis_security/xf_security/types.hpp	58	1,610	0	0	0	2	0	0	Apache-2.0
Vitis_security/xf_security/utils.hpp	44	1,270	0	0	0	5	0	0	Apache-2.0
Vitis_solver/cholesky.hpp	732	31,537	9	19	8	1	8	17	Apache-2.0
Vitis_solver/qrf.hpp	889	29,856	11	22	5	1	24	22	Apache-2.0
Vitis_solver/svd.hpp	1,577	72,409	11	19	1	1	15	44	Apache-2.0
Vitis_solver_L2/potrf.hpp	200	5,967	5	5	0	1	10	24	Apache-2.0
finn_hlslib/activations.hpp	387	14,452	14	21	19	1	1	15	BSD-3-Clause
finn_hlslib/bnn-library.h	60	2,612	0	0	0	0	0	0	BSD-3-Clause
finn_hlslib/convlayer.h	311	16,354	12	3	0	1	0	3	BSD-3-Clause
finn_hlslib/dma.h	238	9,463	6	10	0	1	0	3	BSD-3-Clause
finn_hlslib/interpret.hpp	307	8,529	13	23	11	1	2	36	BSD-3-Clause
finn_hlslib/mac.hpp	206	8,379	2	6	0	1	0	10	BSD-3-Clause
finn_hlslib/maxpool.h	609	23,874	12	10	0	1	0	35	BSD-3-Clause
finn_hlslib/mmv.hpp	59	2,449	0	1	1	1	0	1	BSD-3-Clause
finn_hlslib/mvau.hpp	310	11,892	2	2	0	1	0	16	BSD-3-Clause
finn_hlslib/slidingwindow.h	2,095	87,820	19	21	0	1	0	66	BSD-3-Clause
finn_hlslib/streamtools.h	1,008	35,734	23	25	4	1	1	24	BSD-3-Clause
finn_hlslib/tmrcheck.hpp	204	9,581	2	2	0	1	0	6	BSD-3-Clause
finn_hlslib/upsample.hpp	182	6,383	3	3	0	1	0	2	BSD-3-Clause
finn_hlslib/utils.hpp	107	4,173	3	4	8	1	0	0	BSD-3-Clause
finn_hlslib/vvau.hpp	276	11,276	2	2	0	1	0	14	BSD-3-Clause
finn_hlslib/weights.hpp	169	5,627	3	3	5	1	2	9	BSD-3-Clause
Total	11,917	482,591	176	216	75	41	64	445	—

Table A2 lists every class and struct definition found, and Table A3 every named loop label. Loop labels are included

because in this corpus they are not decoration: they are the handles by which a directive or a synthesis report refers to a loop.

Table A2. All class and struct definitions.

File	Definitions
aes.hpp	aesTable, aesEnc, aesEnc, aesEnc, aesEnc, aesDec, aesDec, aesDec, aesDec
sha224_256.hpp	SHA256Block, sha256_digest_config, sha256_digest_config, sha256_digest_config
cholesky.hpp	choleskyTraits, choleskyTraits, choleskyTraits, choleskyTraits, choleskyTraits, choleskyTraits, InputType, OutputType
qrf.hpp	qrfTraits, is_cmplx, is_cmplx, is_cmplx, qrf_alt_config
svd.hpp	svdTraits
activations.hpp	greater, less, greater_equal, less_equal, add, mul, max, greater, less, greater_equal, less_equal, add, mul, max, Activation, PassThroughActivation, ThresholdActivation, ThresholdsActivation, ChannelWiseOperation
interpret.hpp	XnorMul, Binary, Identity, Recast, Container, Caster, Caster, Slice, Container, Slice_mmv, Container
mmv.hpp	MultiChanData
streamtools.h	WidthAdjustedInputStream, WidthAdjustedInputStream, WidthAdjustedOutputStream, WidthAdjustedOutputStream
utils.hpp	first_param, first_param, ap_resource_dflt, ap_resource_lut, ap_resource_dsp, ap_resource_lutram, ap_resource_bram, ap_resource_uram
weights.hpp	BinaryWeights, TileIndex, FixedPointWeights, TileIndex, Weights_Tile

Table A3. All named loop labels.

File	Labels
aes.hpp	public
cholesky.hpp	col_loop, default, diag_loop, off_diag_loop, row_loop, sum_loop, zero_cols_loop, zero_rows_loop
qrf.hpp	calc_rotations, check_col_indices, col_assign_loop, col_copy_q_i, col_copy_r_i, default, init_counts, init_indices, px, qrf_col_loop, qrf_in_col_assign_Qi, qrf_in_col_assign_Ri, qrf_in_row_assign, qrf_out_col_assign, qrf_out_row_assign, qrf_q_update, qrf_r_update, qrf_row_loop, read_indices, rotate, row_assign_loop, row_copy, update_q, update_r
svd.hpp	col_load, col_loop, col_store, default, diag_index_update, diag_px, off_col, off_diag_loop, off_diag_px, off_row, row_load, row_loop, row_store, step_loop, sweep_loop
potrf.hpp	Loop_add_1, Loop_add_2, Loop_add_3, Loop_col, Loop_per_Unit, Loop_read, Loop_row, Loop_vec_mul, Loop_vec_mul_jj, Loop_write
activations.hpp	public
interpret.hpp	private, public
streamtools.h	public
weights.hpp	private, public

Table A4. All preprocessor macro definitions.

File	Macros
vt_fft.hpp	_VT_FFT_H_
aes.hpp	_XF_SECURITY_AES_HPP_
sha224_256.hpp	BSIG0, BSIG1, CH, MAJ, ROTL, ROTR, SHR, SSIG0, SSIG1, _DEBUG, _XF_SECURITY_PRINT, _XF_SECURITY_SHA224_256_HPP_, _XF_SECURITY_VOID_CAST
types.hpp	_XF_SECURITY_TYPES_HPP_, __int8_t_defined
utils.hpp	XF_SECURITY_ASSERT, XF_SECURITY_MACRO_QUOTE, XF_SECURITY_MACRO_STR, XF_SECURITY_STATIC_ASSERT, _XF_SECURITY_UTILS_HPP_
cholesky.hpp	_XF_SOLVER_CHOLESKY_HPP_
qrf.hpp	_XF_SOLVER_QRF_HPP_
svd.hpp	_XF_SOLVER_SVD_HPP_
potrf.hpp	_XF_SOLVER_POTRF_HPP_
activations.hpp	ACTIVATIONS_HPP
convlayer.h	CONVLAYER_H
dma.h	DMA_HPP
interpret.hpp	INTERPRET_HPP
mac.hpp	MAC_HPP
maxpool.h	MAXPOOL_H
mmv.hpp	MMVCLASS_H
mvau.hpp	MVAU_HPP
slidingwindow.h	SLIDINGWINDOW_H
streamtools.h	STREAMTOOLS_H
tmrcheck.hpp	TMR_HPP
upsample.hpp	UPSAMPLE_HPP
utils.hpp	UTILS_HPP
vvau.hpp	VVAU_HPP
weights.hpp	WEIGHTS_HPP

Table A5. All include directives (the dependency closure of Section II-A).

File	Includes
vt_fft.hpp	vitis_fft/hls_ssr_fft.hpp
aes.hpp	ap_int.h, hls_stream.h, iostream
sha224_256.hpp	ap_int.h, cstdio, hls_stream.h, xf_security/types.hpp, xf_security/utils.hpp
types.hpp	cstdint, stdint.h
utils.hpp	cassert
cholesky.hpp	ap_fixed.h, complex, hls_stream.h, hls_x_complex.h, utils/std_complex_utils.h, utils/x_matrix_utils.hpp
qrf.hpp	complex, hls_math.h, hls_stream.h, hls_x_complex.h, stdio.h, stdlib.h, utils/std_complex_utils.h, utils/x_matrix_utils.hpp
svd.hpp	ap_fixed.h, assert.h, complex, hls_stream.h, hls_x_complex.h, utils/std_complex_utils.h, utils/x_matrix_utils.hpp
potrf.hpp	float.h, hw/math_helper.hpp
activations.hpp	functional, hls_stream.h, interpret.hpp
bnn-library.h	ap_int.h, convlayer.h, dma.h, hls_stream.h, iostream, maxpool.h, mmv.hpp, slidingwindow.h, streamtools.h, string, upsample.hpp, vvau.hpp, weights.hpp
convlayer.h	ap_int.h, hls_stream.h, mvau.hpp, slidingwindow.h, streamtools.h, tmrcheck.hpp
dma.h	ap_int.h, hls_stream.h
interpret.hpp	ap_int.h, ostream
mac.hpp	utils.hpp
maxpool.h	interpret.hpp, limits, utils.hpp
mmv.hpp	hls_stream.h
mvau.hpp	hls_stream.h, interpret.hpp, mac.hpp, weights.hpp
slidingwindow.h	algorithm, utils.hpp
streamtools.h	ap_axi_sdata.h
tmrcheck.hpp	hls_stream.h
upsample.hpp	ap_int.h, hls_stream.h
utils.hpp	ap_int.h, cstdint, fstream, iostream
vvau.hpp	hls_stream.h, interpret.hpp, mac.hpp
weights.hpp	ap_int.h, array

APPENDIX B COMPLETE PRAGMA INDEX

All 445 #pragma HLS directives in the corpus, by file and line. This table is the evidence for every pragma count quoted in the body of the paper.

Vitis_security/aes.hpp — 45 directives

Line	Directive
15	inline
16	resource variable = ssbox core = ROM_nP_LUTRAM
17	resource variable = iibox core = ROM_nP_LUTRAM
67	inline
68	ARRAY_PARTITION variable = key_list complete dim = 1
72	inline off
78	pipeline II = 1
79	pipeline II = 1
138	unroll
164	unroll
172	unroll
205	inline off
213	pipeline II = 1
246	inline
247	ARRAY_PARTITION variable = key_list complete dim = 1
251	inline off
259	pipeline II = 1
315	unroll
341	unroll
349	unroll
387	inline
388	ARRAY_PARTITION variable = key_list complete dim = 1
392	inline off
397	pipeline II = 1
442	unroll
470	unroll
477	unroll
540	inline
541	ARRAY_PARTITION variable = key_list complete dim = 1
545	inline off
551	pipeline II = 1
658	unroll
672	unroll
703	inline
704	ARRAY_PARTITION variable = key_list complete dim = 1
708	inline off
716	pipeline II = 1
819	unroll
833	unroll
864	inline
865	ARRAY_PARTITION variable = key_list complete dim = 1

869	inline off
874	pipeline II = 1
968	unroll
983	unroll

Vitis_security/sha224_256.hpp — 53 directives

Line	Directive
102	pipeline II = 16
103	loop_tripcount min = 0 max = 1
106	array_partition variable = b0.M complete
110	unroll
134	array_partition variable = b.M complete
141	unroll
157	array_partition variable = b.M complete
161	pipeline
210	array_partition variable = b.M complete
214	unroll
255	array_partition variable = b1.M complete
258	unroll
306	pipeline II = 16
307	loop_tripcount min = 0 max = 1
310	array_partition variable = b0.M complete
315	unroll
348	array_partition variable = b.M complete
355	unroll
372	array_partition variable = b.M complete
376	pipeline
476	array_partition variable = b1.M complete
479	unroll
507	loop_tripcount min = 1 max = 1 avg = 1
508	pipeline II = 1
531	latency max = 65
534	array_partition variable = blk.M complete
538	array_partition variable = W complete
542	pipeline II = 1
550	pipeline II = 1
629	array_partition variable = K complete
639	array_partition variable = H complete
664	loop_tripcount min = 1 max = 1
665	latency max = 65
683	pipeline II = 1
703	unroll
719	unroll
752	DATAFLOW
755	STREAM variable = blk_strm depth = 32
756	RESOURCE variable = blk_strm core = FIFO_LUTRAM
760	STREAM variable = nblk_strm depth = 32
761	RESOURCE variable = nblk_strm core = FIFO_LUTRAM
763	STREAM variable = nblk_strm1 depth = 32
764	RESOURCE variable = nblk_strm1 core = FIFO_LUTRAM
766	STREAM variable = nblk_strm2 depth = 32
767	

	RESOURCE variable = nblk_strm2 core = FIFO_LUTRAM
771	STREAM variable = end_nblk_strm depth = 32
772	RESOURCE variable = end_nblk_strm core = FIFO_LUTRAM
774	STREAM variable = end_nblk_strm1 depth = 32
775	RESOURCE variable = end_nblk_strm1 core = FIFO_LUTRAM
777	STREAM variable = end_nblk_strm2 depth = 32
778	RESOURCE variable = end_nblk_strm2 core = FIFO_LUTRAM
782	STREAM variable = w_strm depth = 32
783	RESOURCE variable = w_strm core = FIFO_LUTRAM

Vitis_solver/cholesky.hpp — 17 directives

Line	Directive
368	PIPELINE II = CholeskyTraits::INNER_II
449	loop_tripcount max = 1 + RowsColsA / 2
461	loop_tripcount max = 1 + RowsColsA / 2
462	PIPELINE II = CholeskyTraits::INNER_II
537	ARRAY_PARTITION variable = A cyclic dim = CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR
538	ARRAY_PARTITION variable = L cyclic dim = CholeskyTraits::UNROLL_DIM factor = CholeskyTraits::UNROLL_FACTOR
539	ARRAY_PARTITION variable = L_internal cyclic dim = CholeskyTraits::UNROLL_DIM factor = \
541	ARRAY_PARTITION variable = square_sum_array cyclic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR
542	ARRAY_PARTITION variable = product_sum_array cyclic dim = 1 factor = CholeskyTraits::UNROLL_FACTOR
575	loop_tripcount max = 1 + RowsColsA / 2
590	LOOP_FLATTEN off
591	PIPELINE II = CholeskyTraits::INNER_II
592	UNROLL FACTOR = CholeskyTraits::UNROLL_FACTOR
656	loop_tripcount max = 1 + RowsColsA / 2
657	PIPELINE
712	PIPELINE
722	PIPELINE

Vitis_solver/qrf.hpp — 22 directives

Line	Directive
618	PIPELINE
627	PIPELINE
661	PIPELINE II = QRF_TRAITS::UPDATE_II
670	PIPELINE II = QRF_TRAITS::UPDATE_II
686	PIPELINE
721	ARRAY_PARTITION variable = q_i cyclic dim = 2 factor = QRF_TRAITS::UNROLL_FACTOR
722	ARRAY_PARTITION variable = r_i cyclic dim = 2 factor = QRF_TRAITS::UNROLL_FACTOR
725	STREAM variable = to_rot depth = RowsA / 2
734	STREAM variable = rotations depth = RowsA / 2
742	LOOP_MERGE force
745	PIPELINE
754	PIPELINE
764	LOOP_TRIPCOUNT min = 1 max = RowsA / 2
765	PIPELINE II = QRF_TRAITS::CALC_ROT_II
788	LOOP_TRIPCOUNT min = 1 max = RowsA / 2
805	LOOP_MERGE force
808	PIPELINE II = QRF_TRAITS::UPDATE_II
809	UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR
818	PIPELINE II = QRF_TRAITS::UPDATE_II
819	UNROLL FACTOR = QRF_TRAITS::UNROLL_FACTOR
829	LOOP_MERGE force
832	PIPELINE

Vitis_solver/svd.hpp — 44 directives

Line	Directive
98	inline off
336	inline
449	inline
646	inline
946	PIPELINE II = SVDTraits::OFF_DIAG_II
975	PIPELINE II = SVDTraits::OFF_DIAG_II
1106	ARRAY_PARTITION variable=diag1_i cyclic dim=2 factor=SVDTraits::UNROLL_FACTOR
1107	ARRAY_PARTITION variable=diag2_i cyclic dim=2 factor=SVDTraits::UNROLL_FACTOR
1108	ARRAY_PARTITION variable=s_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1109	ARRAY_PARTITION variable=s_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1110	ARRAY_PARTITION variable=u_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1111	ARRAY_PARTITION variable=u_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1112	ARRAY_PARTITION variable=v_col1 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1113	ARRAY_PARTITION variable=v_col2 cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1114	ARRAY_PARTITION variable=diag_w_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1115	ARRAY_PARTITION variable=diag_x_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1116	ARRAY_PARTITION variable=diag_y_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1117	ARRAY_PARTITION variable=diag_z_out cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1118	ARRAY_PARTITION variable=uw_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1119	ARRAY_PARTITION variable=ux_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1120	ARRAY_PARTITION variable=uy_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1121	ARRAY_PARTITION variable=uz_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1122	ARRAY_PARTITION variable=vw_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1123	ARRAY_PARTITION variable=vx_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1124	ARRAY_PARTITION variable=vy_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1125	ARRAY_PARTITION variable=vz_new cyclic dim=1 factor=SVDTraits::UNROLL_FACTOR
1135	PIPELINE
1166	UNROLL FACTOR = SVDTraits::UNROLL_FACTOR
1185	UNROLL FACTOR = SVDTraits::DIAG_UNROLL_FACTOR
1186	PIPELINE II = SVDTraits::DIAG_II
1234	UNROLL FACTOR = SVDTraits::UNROLL_FACTOR
1251	PIPELINE II = SVDTraits::OFF_DIAG_II

1253	INLINE recursive
1260	dependence variable=s_col1 inter false
1261	dependence variable=s_col2 inter false
1262	dependence variable=u_col1 inter false
1263	dependence variable=u_col2 inter false
1264	dependence variable=v_col1 inter false
1265	dependence variable=v_col2 inter false
1477	PIPELINE
1549	PIPELINE
1557	PIPELINE
1563	PIPELINE
1569	PIPELINE

Vitis_solver_L2/potrff.hpp — 24 directives

Line	Directive
37	loop_tripcount min = 1 max = 8
39	BIND_STORAGE variable = tmp_i type = ram_2p impl = lutram
43	loop_tripcount min = 1 max = N
44	pipeline
45	dependence variable = tmp_i inter false
46	dependence variable = dataA inter false
52	pipeline
58	pipeline
64	pipeline
77	BIND_STORAGE variable = tmp type = ram_2p impl = lutram
81	pipeline
82	dependence variable = tmp inter false
83	dependence variable = dataA inter false
87	unroll
94	pipeline
100	pipeline
106	pipeline
131	DATAFLOW
135	unroll factor = NCU
143	array_partition variable = dataj cyclic factor = NCU
175	array_partition variable = matA cyclic factor = NCU
176	BIND_STORAGE variable = matA type = xpm_memory impl = uram
181	pipeline
191	pipeline

finn_hlslib/activations.hpp — 15 directives

Line	Directive
145	<code>inline</code>
163	<code>inline</code>
180	<code>inline</code>
185	<code>inline</code>
208	<code>inline</code>
214	<code>inline</code>
217	<code>unroll</code>
247	<code>inline</code>
252	<code>inline</code>
296	<code>pipeline style=flp II=1</code>
302	<code>UNROLL</code>
352	<code>ARRAY_PARTITION</code> <code>variable=internal_thr.m_thresholds complete</code> <code>dim=0</code>
358	<code>pipeline style=flp II=1</code>
371	<code>UNROLL</code>
376	<code>UNROLL</code>

finn_hlslib/convlayer.h — 3 directives

Line	Directive
113	<code>INLINE</code>
198	<code>INLINE</code>
284	<code>INLINE</code>

finn_hlslib/dma.h — 3 directives

Line	Directive
141	<code>pipeline style=flp II=1</code>
154	<code>pipeline style=flp II=1</code>
224	<code>pipeline style=flp II=1</code>

finn_hlslib/interpret.hpp — 36 directives

Line	Directive
62	<code>inline</code>
67	<code>inline</code>
72	<code>inline</code>
80	<code>inline</code>
89	<code>inline</code>
97	<code>inline</code>
102	<code>inline</code>
116	<code>inline</code>
122	<code>inline</code>
138	<code>inline</code>
143	<code>inline</code>
147	<code>inline</code>
151	<code>inline</code>
155	<code>inline</code>
163	<code>inline</code>
168	<code>inline</code>
173	<code>inline</code>
204	<code>inline</code>
207	<code>inline</code>
211	<code>inline</code>
215	<code>inline</code>
220	<code>inline</code>
224	<code>inline</code>
232	<code>inline</code>
237	<code>inline</code>
242	<code>inline</code>
261	<code>inline</code>
264	<code>inline</code>
267	<code>inline</code>
271	<code>inline</code>
275	<code>inline</code>
279	<code>inline</code>
284	<code>inline</code>
292	<code>inline</code>
297	<code>inline</code>
302	<code>inline</code>

finn_hlslib/mac.hpp — 10 directives

Line	Directive
89	inline
114	inline
116	BIND_OP variable=res op=mul impl=fabric
140	inline
142	BIND_OP variable=res op=mul impl=dsp
165	inline
168	unroll
192	inline
195	unroll
202	inline

finn_hlslib/maxpool.h — 35 directives

Line	Directive
74	UNROLL
83	pipeline style=flp II=1
91	pipeline style=flp II=1
146	ARRAY_PARTITION variable=buf complete dim=2
149	UNROLL
159	pipeline style=flp II=1
162	UNROLL
176	UNROLL
210	INLINE
255	ARRAY_PARTITION variable=buf complete dim=2
258	pipeline style=flp II=1
260	UNROLL
272	pipeline style=flp II=1
275	UNROLL
288	pipeline style=flp II=1
290	UNROLL
302	pipeline style=flp II=1
331	INLINE
370	pipeline style=flp II=1
373	UNROLL
415	bind_storage variable=accumulators type=RAM_2P impl=LUTRAM
421	pipeline style=flp II=1
425	UNROLL
477	ARRAY_PARTITION variable=topval complete dim=1
479	ARRAY_PARTITION variable=toplabels complete dim=1
484	UNROLL
489	pipeline style=flp II=1
492	UNROLL
501	UNROLL
508	UNROLL
570	ARRAY_PARTITION variable=accu complete dim=0
576	pipeline style=flp II=1
583	UNROLL
590	UNROLL
599	UNROLL

finn_hlslib/mmv.hpp — 1 directives

Line	Directive
53	inline

finn_hlslib/mvau.hpp — 16 directives

Line	Directive
110	ARRAY_PARTITION variable=inputBuf complete dim=0
114	ARRAY_PARTITION variable=accu complete dim=0
124	pipeline style=flp II=1
140	UNROLL
142	UNROLL
151	UNROLL
165	UNROLL
167	UNROLL
232	ARRAY_PARTITION variable=inputBuf complete dim=1
235	ARRAY_PARTITION variable=accu complete dim=0
238	ARRAY_PARTITION variable=w.m_weights complete dim=0
250	pipeline style=flp II=1
267	UNROLL
274	UNROLL
281	UNROLL
294	UNROLL

finn_hlslib/slidingwindow.h — 66 directives

Line	Directive
74	BIND_STORAGE variable=inputBuf type=RAM_2P
95	BIND_STORAGE variable=inputBuf type=RAM_S2P impl=BRAM
116	BIND_STORAGE variable=inputBuf type=RAM_S2P impl=URAM
137	BIND_STORAGE variable=inputBuf type=RAM_S2P impl=LUTRAM
177	ARRAY_PARTITION variable=inputBuf complete dim=1
192	pipeline style=flp II=1
308	DEPENDENCE variable=inputBuf inter false
309	DEPENDENCE variable=inputBuf intra false
310	ARRAY_PARTITION variable=inputBuf complete dim=1
311	ARRAY_PARTITION variable=inputBuf complete dim=2
326	pipeline style=flp II=1
333	UNROLL
359	UNROLL
392	UNROLL
466	ARRAY_PARTITION variable=inputBuf complete dim=1
467	DEPENDENCE variable=inputBuf inter false
468	DEPENDENCE variable=inputBuf intra false
476	DEPENDENCE variable=current_block_write intra false
479	pipeline style=flp II=1
544	DEPENDENCE variable=inputBuf inter false
545	DEPENDENCE variable=inputBuf intra false
554	DEPENDENCE variable=current_block_write intra false
610	DEPENDENCE variable=inputBuf inter false
611	DEPENDENCE variable=inputBuf intra false
612	ARRAY_PARTITION variable=inputBuf complete dim=1
613	ARRAY_PARTITION variable=inputBuf complete dim=2
628	DEPENDENCE variable=current_block_write intra false
631	pipeline style=flp II=1
638	UNROLL
672	UNROLL
705	UNROLL
716	DEPENDENCE variable=current_block_write intra false
767	ARRAY_PARTITION variable=inputBuf complete dim=1
782	pipeline style=flp II=1
903	ARRAY_PARTITION variable=inputBuf complete dim=1
904	DEPENDENCE variable=inputBuf inter false

905	DEPENDENCE variable=inputBuf intra false
915	pipeline style=flp II=1
917	DEPENDENCE variable=current_block_write intra false
983	DEPENDENCE variable=inputBuf inter false
984	DEPENDENCE variable=inputBuf intra false
993	DEPENDENCE variable=current_block_write intra false
1048	DEPENDENCE variable=inputBuf inter false
1049	DEPENDENCE variable=inputBuf intra false
1050	ARRAY_PARTITION variable=inputBuf complete dim=1
1051	ARRAY_PARTITION variable=inputBuf complete dim=2
1066	pipeline style=flp II=1
1073	UNROLL
1099	UNROLL
1132	UNROLL
1191	performance target_ti=IFMDim*IFMDim*IFMChannels/SIMD
1198	pipeline style=flp II=IFMChannels/SIMD
1245	pipeline style=flp II=1
1300	ARRAY_PARTITION variable=inputBuf complete dim=1
1315	pipeline style=flp II=1
1434	ARRAY_PARTITION variable=inputBuf complete dim=1
1449	pipeline style=flp II=1
1575	ARRAY_PARTITION variable=inputBuf complete dim=1
1590	pipeline style=flp II=1
1710	ARRAY_PARTITION variable=inputBuf complete
1718	pipeline style=flp II=1
1739	UNROLL
1824	pipeline style=flp II=1
1908	pipeline style=flp II=1
1984	pipeline style=flp II=1
2063	pipeline style=flp II=1

finn_hlslib/streamtools.h — 24 directives

Line	Directive
77	pipeline style=flp II=1
160	pipeline style=flp II=1
234	pipeline style=flp II=1
280	pipeline style=flp II=1
374	inline
451	pipeline style=flp II=1
471	pipeline style=flp II=1
482	pipeline style=flp II=1
591	pipeline style=flp II=1
649	pipeline style=flp II=1
654	UNROLL
723	INLINE
770	pipeline style=flp II=1
777	UNROLL
793	pipeline style=flp II=1
799	UNROLL
811	pipeline style=flp II=1
815	UNROLL
851	pipeline style=flp II=1
855	UNROLL
882	pipeline style=flp II=1
886	UNROLL
975	pipeline style=flp II=1
998	pipeline style=flp II=1

finn_hlslib/tmrcheck.hpp — 6 directives

Line	Directive
82	ARRAY_PARTITION variable=red_ch_index complete dim=0
91	pipeline style=flp II=1
99	ARRAY_PARTITION variable=numerrors complete dim=0
103	UNROLL
129	ARRAY_PARTITION variable=compute complete dim=0
132	UNROLL

finn_hlslib/upsample.hpp — 2 directives

Line	Directive
86	pipeline style=flp II=1
175	pipeline II=1 style=flp

finn_hlslib/vvau.hpp — 14 directives

Line	Directive
103	ARRAY_PARTITION variable=accu complete dim=0
112	pipeline style=flp II=1
119	UNROLL
128	UNROLL
142	UNROLL
144	UNROLL
213	ARRAY_PARTITION variable=accu complete dim=0
223	pipeline style=flp II=1
230	UNROLL
239	ARRAY_PARTITION variable=w.m_weights complete dim=0
241	UNROLL
246	UNROLL
260	UNROLL
262	UNROLL

finn_hlslib/weights.hpp — 9 directives

Line	Directive
83	inline
88	inline
95	inline
127	inline
132	inline
135	unroll
145	inline
157	inline
160	unroll

APPENDIX C COMPLETE FUNCTION INDEX

All 176 function definitions located by the extraction pass, by file and line, with return type. Template parameter lists are omitted for space; they appear in the source at the line given.

Vitis_security/aes.hpp — 8 definitions

Line	Returns	Name
13	public:	aesTable
56	public: void	updateKey
58	void	process
70	void	updateKey
114	ap_uint<8>	GFMul2
203	void	updateKey_2
595	ap_uint<8>	GFMul4
609	ap_uint<8>	GFMul8

Vitis_security/sha224_256.hpp — 16 definitions

Line	Returns	Name
82	void	preProcessing
497	void	dup_strm
522	void	generateMsgSchedule
564	void	sha256_iter
609	template <int h_width> void	sha256Digest
746	template <int m_width, int h_width> inline void	sha256_top
754	hls::stream<SHA256Block>	blk_strm
759	hls::stream<uint64_t>	nblk_strm
762	hls::stream<uint64_t>	nblk_strm1
765	hls::stream<uint64_t>	nblk_strm2
770	hls::stream<bool>	end_nblk_strm
773	hls::stream<bool>	end_nblk_strm1
776	hls::stream<bool>	end_nblk_strm2
781	hls::stream<uint32_t>	w_strm
809	template <int m_width> void	sha224
826	template <int m_width> void	sha256

Vitis_solver/cholesky.hpp — 9 definitions

Line	Returns	Name
213	template <typename T_IN, typename T_OUT> int	cholesky_sqrt_op
256	template <typename InputType, typename OutputType> void	cholesky_rsqr
261	template <int W1, int I1, ap_q_mode Q1, ap_o_mode O1, int...	cholesky_rsqr
275	template <typename AType, typename BType, typename CType>...	cholesky_prod_sum_mult
295	template <bool LowerTriangularL, int RowsColsA, typename ...>	choleskyBasic
412	template <bool LowerTriangularL, int RowsColsA, typename ...>	choleskyAlt
515	template <bool LowerTriangularL, int RowsColsA, typename ...>	choleskyAlt2
673	template <bool LowerTriangularL, int RowsColsA, typename ...>	choleskyTop
707	int	cholesky

Vitis_solver/qrf.hpp — 11 definitions

Line	Returns	Name
69	template <typename T> int	qrf_require_extra_pass
97	template <class T> bool	is_zero
124	template <typename T> T	qrf_vm
130	template <typename T> void	qrf_mm
140	template <typename T> void	qrf_mm_or_mag
161	template <typename T> T	qrf_magnitude
236	return largest *	x_sqrt
306	template <typename T> void	qrf_givens
587	template <bool TransposedQ, int RowsA, int ColsA, typename...	qrf_basic
702	template <bool TransposedQ, int RowsA, int ColsA, typename...	qrf_alt
868	void	qrf

Vitis_solver/svd.hpp — 11 definitions

Line	Returns	Name
69	template <typename T> bool	within_precision
85	template <typename AType, typename BType, typename CType>...	vm2x1
95	template <typename AType, typename BType, typename CType>...	vm2x1_base
101	template <typename T> void	vm2x1
136	template <typename AType, typename BType, typename CType>...	mm2x2
156	template <typename InType, typename CType> void	calc_angle
316	template <typename AInType, typename CType, typename AOu...	svd2x2
821	template <int RowsA, int ColsA, class SVDTraits, typename...	svdBasic
1037	template <int RowsA, int ColsA, class SVDTraits, typename...	svdPairs
1503	template <int RowsA, int ColsA, class SVDTraits, typename...	svdTop
1539	void	svd

Vitis_solver_L2/potrf.hpp — 5 definitions

Line	Returns	Name
32	template <typename T, int N, int NCU> void	chol_col
73	template <typename T, int N, int NCU> void	chol_jj
128	template <typename T, int N, int NCU> void	chol_col_wrapper
139	template <typename T, int N, int NCU> void	cholesky_core
168	template <typename T, int NMAX, int NCU> void	potrf

finn_hlslib/activations.hpp — 14 definitions

Line	Returns	Name
85	ap_uint<1>	operator
113	result_type	operator
143	public: TA	init
146	return	TA
152	T0	activate
161	public: T	activate
178	public:	ThresholdActivation
182	public: bool	activate
186	return	Compare
211	public: TR	activate
245	public: TI	init
248	return	TI
281	typename TI, typename T0, typename TA> void	Thresholding_Batch
340	typename TI, typename T0> void	Thresholding_Stream_Batch

finn_hlslib/convlayer.h — 12 definitions

Line	Returns	Name
105	typename TW, typename TA, typename R > void	ConvLayer
117	hls::stream<ap_uint<SIMD*TSrcI::width> >	wa_in
118	hls::stream<ap_uint<SIMD*TSrcI::width> >	convInp
119	hls::stream<ap_uint<PE*TDstI::width> >	mvOut
187	typename TW, typename TA, typename R > void	ConvLayer
206	hls::stream<ap_uint<OFMChannels*TDstI::width> >	tmr_in
276	typename TW, typename TA, typename R > void	ConvLayer
289	hls::stream<ap_uint<SIMD * TSrcI::width> >	wa_in
291	hls::stream<MultiChanData<MMV, SIMD * TSrcI::width> >	convInp
292	hls::stream<MultiChanData<MMV, PE * TDstI::width> >	mmv2dwc
293	hls::stream<MultiChanData<MMV, OFMChannels * TDstI::width>>	dwc2flat
294	hls::stream<ap_uint<MMV * OFMChannels * TDstI::width> >	mvOut

finn_hlslib/dma.h — 6 definitions

Line	Returns	Name
64	template<unsigned int DataWidth, unsigned int numBytes> void	Mem2Stream
80	template<unsigned int DataWidth, unsigned int numBytes> void	Mem2Stream_Batch
93	template<unsigned int DataWidth, unsigned int numBytes> void	Stream2Mem
109	template<unsigned int DataWidth, unsigned int numBytes> void	Mem2Stream_Batch_external_wmem
132	template<unsigned int DataWidth, unsigned int numBytes> void	Stream2Mem_Batch
214	template<unsigned int TILES, unsigned int SIMD, unsigned...>	GenParamStream

finn_hlslib/interpret.hpp — 13 definitions

Line	Returns	Name
60	public:	XnorMul
79	explicit	Binary
119	template<typename T> T	operator
123	return	T
136	public:	Container
150	T	operator
159	public: template<typename TV> Container<TV>	operator
166	template<typename TV> Container<TV>	operator
180	template<int M> static T	cast
186	template<int M> static ap_fixed<W, I, Q, O, N>	cast
209	public: auto	operator
212	return	m_val
274	auto	operator

finn_hlslib/mac.hpp — 2 definitions

Line	Returns	Name
87	template<typename TC, typename TD> auto	mul
163	template<unsigned N, typename T, typename TC, typename TD...	mac

finn_hlslib/maxpool.h — 12 definitions

Line	Returns	Name
67	template<unsigned int ImgDim, unsigned int PoolDim, unsig...	StreamingMaxPool
113	template<unsigned int ImgDim, unsigned int PoolDim, unsig...	StreamingMaxPool_Batch
138	template<unsigned int ImgDim, unsigned int PoolDim, unsig...	StreamingMaxPool_Precision
208	void	StreamingMaxPool_Precision_Batch
213	hls::stream<ap_uint<NumChannels*ActType::width>>	wa_in
214	hls::stream<ap_uint<NumChannels*ActType::width>>	mvOut
243	template<unsigned int ImgDim, unsigned int PoolDim, unsig...	StreamingMaxPool_Precision_Batch
326	template<unsigned int ImgDim, unsigned int PoolDim, unsig...	StreamingMaxPool_Precision_Batch
360	void	AccPool_Batch
405	template< unsigned int ImgDim, unsigned int NumChannels, ...	hls::stream<ap_uint<SIMD*Input_precision>> & in, hls::st...
461	unsigned int NumClasses, unsigned int PECount, unsigned i...	LabelSelectBatch
558	typename TI, typename TO, typename TA > void	Pool_batch

finn_hlslib/mvau.hpp — 2 definitions

Line	Returns	Name
91	typename TI, typename TO, typename TW, typename TA, typen...	Matrix_Vector_Activate_Batch
213	typename TI, typename TO, typename TA, typename R > void	Matrix_Vector_Activate_Stream_Batch

finn_hlslib/slidingwindow.h — 19 definitions

Line	Returns	Name
72	template <typename T> void	memory_resou
159	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
287	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
440	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
587	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
749	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
877	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
1027	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
1177	template< unsigned int IFMChannels, unsigned int Input_pr...	ConvolutionI
1227	template< unsigned int IFMChannels, unsigned int Input_pr...	ConvolutionI
1278	template<unsigned int ConvKernelDim_x, unsigned int ConvK...	ConvolutionI
1412	template<unsigned int ConvKernelDim_x, unsigned int ConvK...	ConvolutionI
1549	template<unsigned int ConvKernelDim_x, unsigned int ConvK...	ConvolutionI
1687	template<unsigned int ConvKernelDim, unsigned int IFMChan...	ConvolutionI
1696	hls::stream<ap_uint<SIMD*Input_precision>> & in, hls::st...	__attribute_
1796	template<unsigned int ConvKernelDim_x, unsigned int IFMCh...	ConvolutionI
1882	void	ConvolutionI
1959	void	ConvolutionI
2037	void	ConvolutionI

finn_hlslib/streamtools.h — 23 definitions

Line	Returns	Name
68	template<unsigned int DataWidth, unsigned int NumAllowed, ...	StreamLimiter
102	template<unsigned int DataWidth, unsigned int NumAllowed, ...	StreamLimiter_Batch
137	void	SameResize
206	void	SameResize_Batch
231	template<typename InT, typename OutT> void	StreamingCast
259	template< unsigned OutputDim_x, unsigned OutputDim_y, uns...	FMPadding_nonsquare
317	template< unsigned OutputDim_x, unsigned OutputDim_y, uns...	FMPadding_nonsquare_Batch
361	template< unsigned ImgDim, unsigned OutputDim, unsigned P...	FMPadding
401	template< unsigned ImgDim, unsigned OutputDim, unsigned P...	FMPadding_Batch
436	template<unsigned int InWidth, unsigned int OutWidth, uns...	StreamingDataWidthConverter_Batch
512	template< unsigned int InWidth, unsigned int OutWidth > void	StreamingDataWidthConverterNoMultipl
584	template<unsigned int DataWidth, unsigned int NumTotal > ...	DuplicateStreams
613	template<unsigned int DataWidth, unsigned int NumTotal > ...	DuplicateStreams_Batch
645	void	AddStreams
689	void	AddStreams_Batch
721	void	AddStreamsLayer_Batch
756	unsigned int NumVecs > void	MultiChanDataWidthConverter_Batch
844		FlattenMultiChanData

Line	Returns	Name
875	template<unsigned int NumChannels, unsigned int DataWidth...> void	PackMultiChanData
897	public:	WidthAdjustedInputStream
930	public:	WidthAdjustedOutputStream
970	template<unsigned int DataWidth, unsigned int NumTotal> void	Qdma2Stream_Batch
994	template<unsigned int DataWidth, unsigned int NumTotal> void	Stream2Qdma_Batch

finn_hlslib/weights.hpp — 3 definitions

finn_hlslib/tmrcheck.hpp — 2 definitions

Line	Returns	Name
69	template<unsigned int InW, unsigned int OFMChannels, unsigned int...> void	TMRCheck
185	template<unsigned int InW, unsigned int OFMChannels, unsigned int...> void	TMRCheck_Batch

finn_hlslib/upsample.hpp — 3 definitions

Line	Returns	Name
62	template<unsigned int OFMDim, unsigned int IFMDim, unsigned int...> void	UpsampleNearestNeighbour
134	template<unsigned int OFMDim, unsigned int IFMDim, unsigned int...> void	UpsampleNearestNeighbour_Batch
164	void	UpsampleNearestNeighbour_1D

finn_hlslib/utils.hpp — 3 definitions

Line	Returns	Name
58	unsigned	clog2
88	template < unsigned int BitWidth > void	logStringStream
90	std::ofstream	ofs

finn_hlslib/vvau.hpp — 2 definitions

Line	Returns	Name
79	public:	TileIndex
92	public: TileIndex	weights
96	return	TileIndex

REFERENCES

1. AMD/Xilinx, *Vitis Libraries — Solver Library*, source files solver/L1/include/hw/cholesky.hpp, qrf.hpp, svd.hpp and solver/L2/include/hw/MatrixDecomposition/potrf.hpp, Apache License 2.0. Retrieved September 18, 2026. [full text read]
2. AMD/Xilinx, *Vitis Libraries — Security Library*, source files security/L1/include/xf_security/aes.hpp, sha224_256.hpp, types.hpp, utils.hpp, Apache License 2.0. Retrieved September 18, 2026. [full text read]
3. Xilinx, *FINN-hlslib*, sixteen source files as listed in Appendix A, BSD 3-Clause licence. Retrieved September 18, 2026. [full text read]
4. Acellera Systems Initiative, *UVM Core*, source files uvm_pkg.sv, uvm_component.svh, uvm_sequence.svh, dpi/uvm_dpi.cc. Consulted for the comparison in Section VI-H. [partial read]
5. lowRISC, *OpenTitan*, AES intellectual-property block, 37 register-transfer-level source files. Consulted to establish the generated-versus-handwritten ratio cited in Section VII. [headers of all 37 files read]
6. OpenHW Group, CVA6, file list core/Flist.cva6. Consulted for the language distribution cited in Section VII. [file list read]
7. IEEE Standard for SystemVerilog, IEEE Std 1800. Referenced for the Direct Programming Interface semantics discussed in Section VI-H. [not read; cited for the standard's existence only]
8. National Institute of Standards and Technology, *Advanced Encryption Standard*, FIPS 197, and *Secure Hash Standard*, FIPS 180-4. Referenced as the specifications implemented by the cryptographic family. [not read in this study]

Note on citation practice. Each reference carries a bracketed statement of how far it was actually read. References 7 and 8 were not consulted during this work and are cited only to name the standards that the surveyed code implements; no claim in this paper depends on their content.